

PATENT US 7502723B1 ANALYSIS

Asymmetric Minor Hysteresis Loop Model and Circuit Simulator Including the Same

Physical meaning, complete algorithm logic, analysis of figures
and description of the Python implementation

Dmitriy Makhnovskiy

FINGO Research Centre, Moscow

September 11, 2026

Contents

1. Purpose of the Report and Source Materials	4
2. What the Engelhardt Patent Actually Solves.....	4
2.1. Context: Jiles-Atherton and Chan Hysteresis Models.....	4
2.2. The Idea Behind the Improvement of the Chan Hysteresis Model	4
3. Mathematical Basis of the Model	5
3.1. Magnetic Core Material Parameters.....	5
3.2. Upper and Lower Branches of the Major Loop and the Initial Magnetization Curve	5
3.3. Ordinary Symmetrical Minor Branches and the B_d Parameter	6
3.4. Basis Functions for the Affine Correction	6
3.5. Affine Transformation.....	6
3.6. Limiting Functions and the Meaning of B_d _turn.....	7
3.7. The H_0 Point.....	7
3.8. Scale and Offset Formulas Used in Python	7
4. Analysis of the Patent Figures	8
4.1. FIG. 1 — Defect of the Chan Model	8
4.2. FIG. 2A — Geometry of Normal and Problematic Reversals.....	8
4.3. FIG. 2B — Result of the New Method.....	10
4.4. FIG. 3A — Algorithm Branch for Increasing H	11
4.5. FIG. 3B — Algorithm Branch for Decreasing H	12
4.6. FIG. 4 — Circuit Simulation Example.....	19
4.7. FIG. 5A and FIG. 5B — Effect of the Model on Inductor Current	19
4.8. FIG. 6 — Computational Implementation.....	20
5. Complete FIG. 3 Algorithm: Steps 300-372	21
5.1. General Steps 300-306.....	21
5.2. Increasing H : Steps 308-316 — Reversal Handling.....	21
5.3. Step 312 — Construction of the Asymmetrical Branch for $H \uparrow$	21
5.4. Steps 318-332 — Formula Selection for $H \uparrow$	21
5.5. Decreasing H : Steps 348-356 — Mirrored Reversal	22
5.6. Steps 358-372 — Formula Selection for $H \downarrow$	22
6. Patent Claims.....	23
7. Python Project for Hysteresis-Loop Calculation.....	23
7.1 Project Structure	23
7.2 BH.py Function Module	24
7.3 main.py Control Loop.....	26
7.4 Test Functions from the Hscan.py Module	27
7.5 Loading Parameters in Parameters.py	27
7.6 Correspondence Between the Patent and the Python Implementation.....	28

7.7 Checks, Tests, and Diagnostic Invariants	28
7.8 Notes on the Current Implementation and Recommendations.....	29
7.9 Calculation Results	29
7.10 Examples of Numerical Hysteresis-Loop Calculations	30
8. Python Project for Calculating Transient Currents in an Inductor with a Magnetic Core.....	32
8.1 Python Project Structure.....	34
8.2 Initial State.....	34
8.3 Trial Calculation of the Magnetic State	35
8.4 Preserving the Hysteresis State During Root Finding	36
8.5 Residual Function	36
8.6 Finding the Bracketing Interval for Bisection	36
8.7 Bisection Method.....	37
8.8 Committing the New State.....	38
8.9 Voltage Source and Parameters	38
8.10 Calculation Results.....	39
8.11 Examples of Inductor Transient-Current Calculations and Comparison with LTspice.....	39
References.....	43

1. Purpose of the Report and Source Materials

This report systematizes the model of asymmetrical minor hysteresis loops proposed by Michael Thomas Engelhardt in U.S. Patent [US 7502723B1](#),[1] and compares it with the current Python implementation of the project. The main objective is to produce a document from which the model logic can be reconstructed independently, the program can be checked against the patent flowchart, and further development can continue without having to reread the patent from scratch.

The patent was granted on March 10, 2009; the application was filed on September 1, 2005. Linear Technology Corporation is listed as the assignee. The subject of the patent is a computationally lightweight model of a ferromagnetic core in an inductor or transformer that can correctly describe asymmetrical minor loops and, in particular, eliminate the nonphysical behaviour of the [Chan](#) model,[2] in which minor branches can extend outside the major loop.

All patent figures in this report are reproduced directly from the original PDF. They have not been redrawn or annotated. Figures that appear sideways in the original patent were only rotated during page layout so that the labels are readable; their content and graphics were not altered.

2. What the Engelhardt Patent Actually Solves

2.1. Context: Jiles-Atherton and Chan Hysteresis Models

In the BACKGROUND section, the patent contrasts two approaches. The [Jiles-Atherton](#) model [3] is presented as physically meaningful and accurate, but computationally more demanding: it uses transcendental functions, is less conveniently parameterized by standard ferromagnetic material data-sheet quantities, and, according to the patent, is more difficult to apply to gapped cores. [The Chan model](#),[2] by contrast, is computationally simple, requires no transcendental functions, and is specified by the familiar parameters H_c , B_r , and B_s , but it has a significant defect for asymmetrical minor loops.[1]

The key problem with Chan arises when, after a field reversal, an ordinary symmetrical minor loop cannot be continued so that it reaches the required turning point. In this situation Chan uses horizontal and vertical translation of a major-loop branch. Geometrically, this can create a branch that lies outside the major loop. Because this outer branch becomes too shallow, the simulator incorrectly interprets the core as being more deeply saturated than it actually is.

2.2. The Idea Behind the Improvement of the Chan Hysteresis Model

Engelhardt retains the simple Chan framework for the major loop and ordinary symmetrical minor loops, but replaces the problematic asymmetrical-branch construction. If a symmetrical minor loop can be correctly extended to the turning point, the ordinary vertically shifted branch is used. If it cannot, a new branch is constructed as an affine transformation of a special basis function. In addition, this branch is bounded by a function associated with the preceding minor branch so that it cannot cross the loop from which the reversal occurred.

3. Mathematical Basis of the Model

3.1. Magnetic Core Material Parameters

In the form given in Table 1 of the patent, the model uses three principal material characteristics:

- B_s — saturation flux density;
- B_r — remanent flux density;
- H_c — coercive field strength.

The formulas also contain the linear vacuum contribution $\mu_0 \cdot H$.

TABLE 1

Functions used:

```

MIN(x, y) = x < y ? x : y
MAX(x, y) = x > y ? x : y
Upper(H) = Bs * (H + Hc) / (|H + Hc| + Hc * (Bs/Br - 1)) + μ0 * H
Lower(H) = Bs * (H - Hc) / (|H - Hc| + Hc * (Bs/Br - 1)) + μ0 * H
Binit(H) = .5 * (Upper(H) + Lower(H))
UpperBmag(H) = H < 0 ? Lower(H) + Br : Binit(H)
LowerBmag(H) = H > 0 ? Upper(H) - Br : Binit(H)
BlatchUpper(H) = scale * UpperBmag(H) + offset
BlatchLower(H) = scale * LowerBmag(H) + offset
BlatchUpperLimit(H) = Upper(H) - Bd_turn
BlatchLowerLimit(H) = Lower(H) + Bd_turn

```

Variables used:

```

Blast, Hlast, OnInitMag, IsLatched, Bd
Bd_turn, Ho, scale, offset
dir (Direction H used to be changing)

```

3.2. Upper and Lower Branches of the Major Loop and the Initial Magnetization Curve

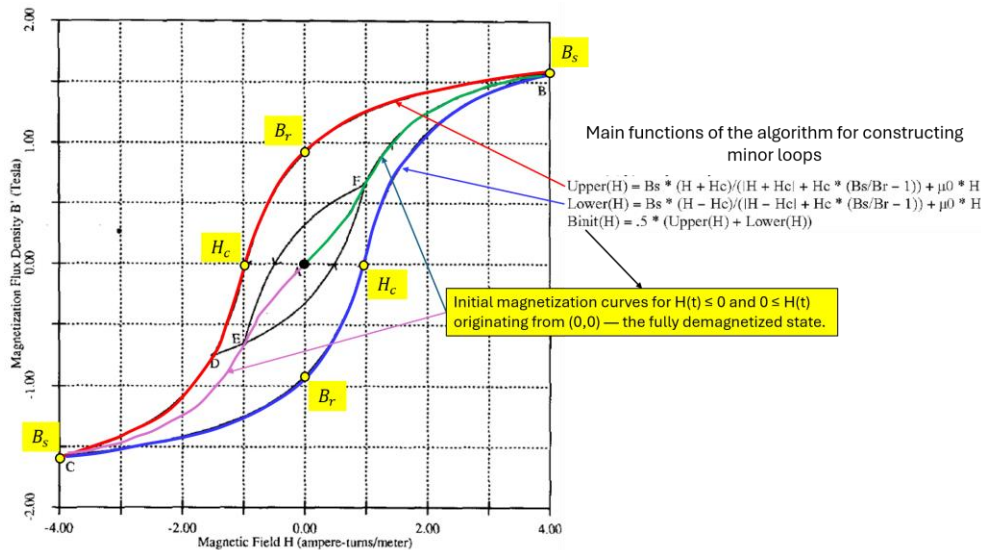
$$\text{Upper}(H) = B_s \cdot (H + H_c) / (|H + H_c| + H_c \cdot (B_s/B_r - 1)) + \mu_0 \cdot H$$

$$\text{Lower}(H) = B_s \cdot (H - H_c) / (|H - H_c| + H_c \cdot (B_s/B_r - 1)) + \mu_0 \cdot H$$

These functions from the Chan model define the two boundaries of the major loop. In particular, by construction, at $H = 0$ the upper branch passes through $+B_r$ and the lower branch through $-B_r$. Thus B_r and H_c are built directly into the geometry of the loop, without requiring a separate numerical solution of a differential equation.

$$\text{Binit}(H) = \frac{1}{2} \cdot [\text{Upper}(H) + \text{Lower}(H)]$$

The initial magnetization curve is defined as the average of the two major-loop branches. For the symmetrical model, $\text{Binit}(0) = 0$. The variable `OnInitMag` in the algorithm does not mean that the model has never undergone a reversal; rather, it indicates that the current trajectory lies on this initial magnetization curve.



3.3. Ordinary Symmetrical Minor Branches and the Bd Parameter

The parameter B_d is the vertical shift between a minor-loop branch and the corresponding branch of the major loop. For an interval with increasing H , the following expression is used:

$$B(H) = \text{Lower}(H) + B_d$$

After a reversal from decreasing H to increasing H , B_d is selected from the continuity condition at the turning point (H_{last} , B_{last}):

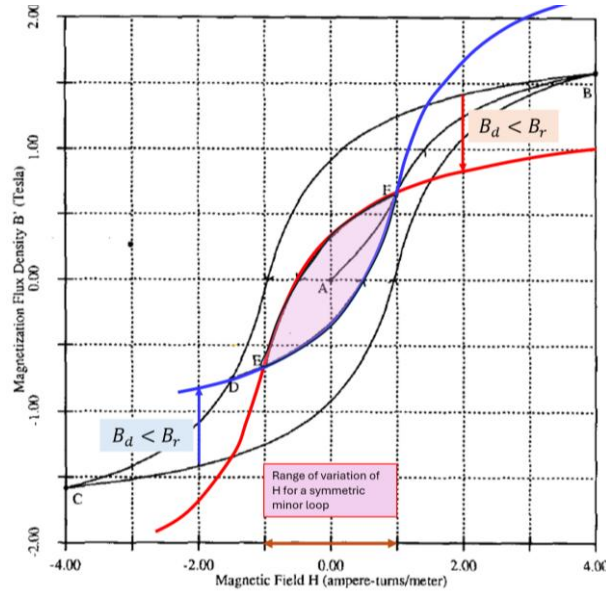
$$B_d = B_{\text{last}} - \text{Lower}(H_{\text{last}})$$

For an interval with decreasing H , the mirrored expression is used:

$$B(H) = \text{Upper}(H) - B_d$$

$$B_d = \text{Upper}(H_{\text{last}}) - B_{\text{last}}$$

These expressions are important: at a turning point, the new branch automatically passes through the already calculated point B_{last} and therefore does not create a discontinuity in B .



3.4. Basis Functions for the Affine Correction

$$\text{UpperBmag}(H) = \text{Lower}(H) + B_r, H < 0; \quad B_{\text{init}}(H), H \geq 0$$

$$\text{LowerBmag}(H) = \text{Upper}(H) - B_r, H > 0; \quad B_{\text{init}}(H), H \leq 0$$

These are composite basis curves. Each connects the initial magnetization curve to a major-loop branch shifted by B_r . The patent uses precisely this form as the template for the problematic asymmetrical minor branch.

3.5. Affine Transformation

$$\text{BlatchUpper}(H) = \text{scale} * \text{UpperBmag}(H) + \text{offset}$$

$$\text{BlatchLower}(H) = \text{scale} * \text{LowerBmag}(H) + \text{offset}$$

The patent states “solve affine map constants gain and offset,” but does not give explicit algebraic expressions for the coefficients. In our implementation they are recovered from the requirement that the transformed basis curve pass through two endpoints: the turning point (H_{last} , B_{last}) and the point (H_0 , $B_{\text{init}}(H_0)$). This is a mathematical specification introduced in the Python code rather than a formula written explicitly in the patent.

3.6. Limiting Functions and the Meaning of Bd_turn

$$\text{BlatchUpperLimit}(H) = \text{Upper}(H) - \text{Bd_turn}$$

$$\text{BlatchLowerLimit}(H) = \text{Lower}(H) + \text{Bd_turn}$$

Bd_turn is the stored value of Bd from the preceding branch at the moment of reversal. It is copied before Bd is recalculated for the new branch. Thus Bd_turn fixes the geometry of the branch from which the trajectory departed and is used as a boundary that the new affine branch must not cross.

3.7. The H0 Point

For increasing H, the point H0 is determined by the equation:

$$\text{Binit}(H0) = \text{Upper}(H0) - \text{Bd_turn}$$

For decreasing H:

$$\text{Binit}(H0) = \text{Lower}(H0) + \text{Bd_turn}$$

Physically, H0 is the field at which the limiting branch intersects the initial magnetization curve. The latched affine solution is used up to H0. Once H0 is reached, the latch is cleared.

3.8. Scale and Offset Formulas Used in Python

For the increasing branch, two points, (H0, B0) and (Hlast, Blast), are used through which the curve UpperBmag(H) must pass after the affine transformation (vertical scaling and vertical offset):

$$\text{scale} * \text{UpperBmag}(H_{\text{last}}) + \text{offset} = \text{Blast}$$

$$\text{scale} * \text{UpperBmag}(H0) + \text{offset} = \text{Binit}(H0)$$

From these two equations, the affine-transformation coefficients are obtained:

$$\text{scale} = [\text{Binit}(H0) - \text{Blast}] / [\text{UpperBmag}(H0) - \text{UpperBmag}(H_{\text{last}})]$$

$$\text{offset} = \text{Blast} - \text{scale} * \text{UpperBmag}(H_{\text{last}})$$

For the decreasing branch, UpperBmag is replaced by LowerBmag. These formulas follow directly from a two-point affine fit and correspond to the patent wording that the basis function is mapped onto the endpoints of the asymmetrical branch.

4. Analysis of the Patent Figures

4.1. FIG. 1 — Defect of the Chan Model

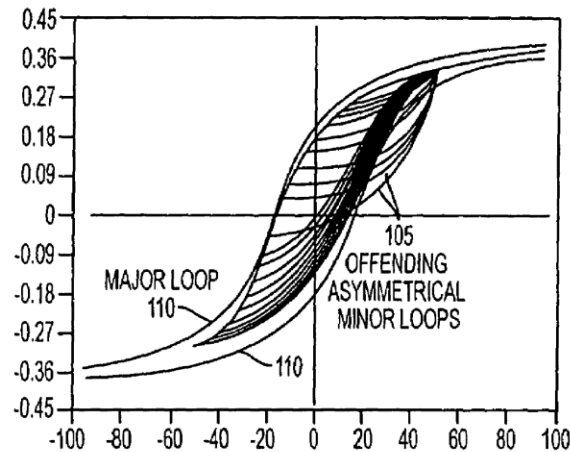


FIG. 1
PRIOR ART

Patent FIG. 1: offending asymmetrical minor loops in the Chan model (prior art).

FIG. 1 illustrates the original problem. The major loop is designated 110, and the group of problematic asymmetrical minor loops is designated 105. Some of these branches extend outside the major loop. In such a region the slope dB/dH becomes anomalously small, which in circuit simulation is equivalent to the false appearance of deep core saturation.

4.2. FIG. 2A — Geometry of Normal and Problematic Reversals

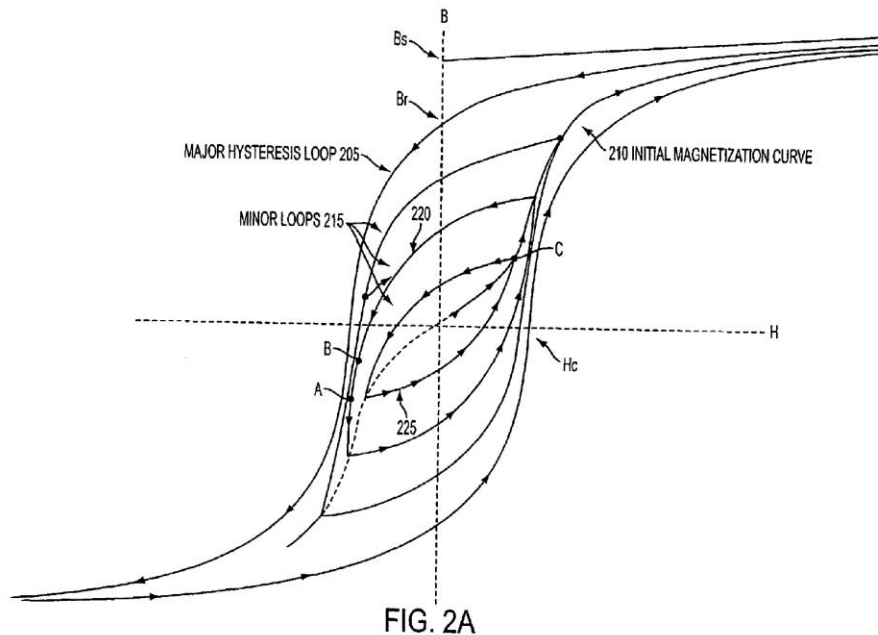


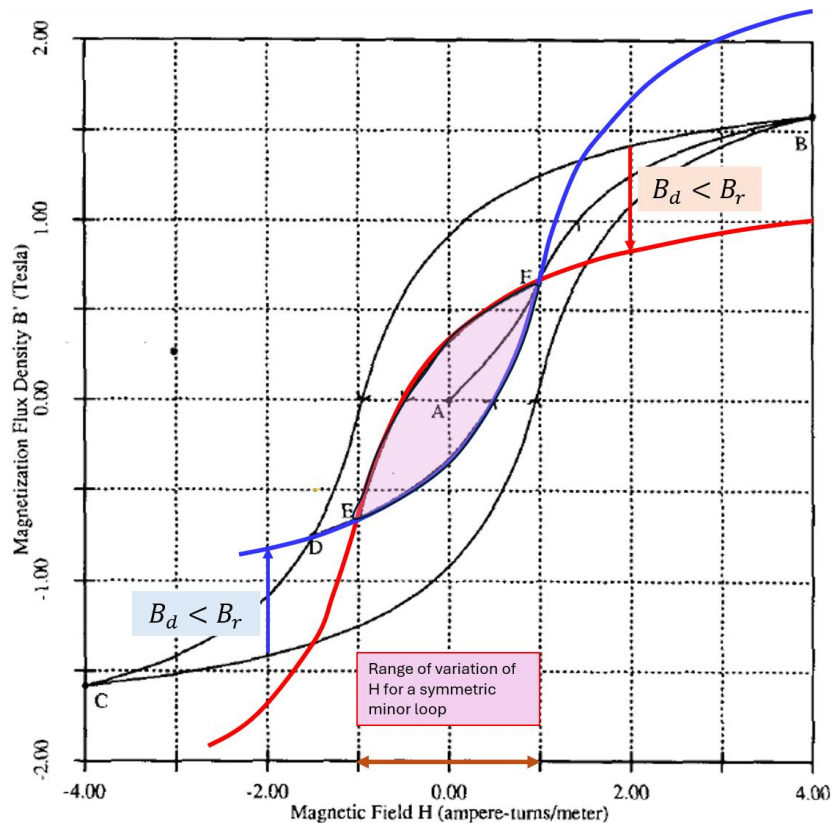
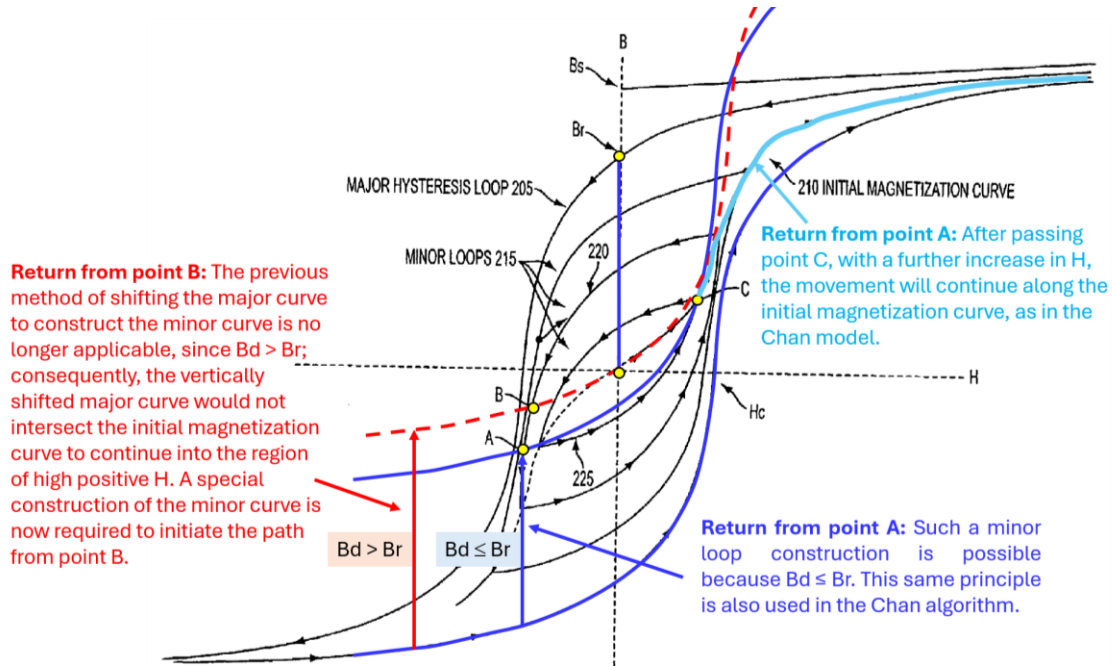
FIG. 2A

Patent FIG. 2A: major loop 205, initial magnetization curve 210, minor loops 215/220/225, and points A, B, and C.

FIG. 2A is the central geometric illustration in the patent. Motion is considered along the upper branch of minor loop 220 in the direction of decreasing H . If the reversal occurs at point A, there exists a minor curve 225 that can be

shifted to point A. The trajectory then joins the initial magnetization curve 210 at point C. The patent allows a derivative discontinuity at C and notes that it does not create a significant problem for simulation.

If the reversal occurs at point B, there is no longer a minor curve that can be shifted to B, because this would require a vertical displacement greater than B_r . This is where the new construction is activated: an affine mapping of a basis function bounded by the preceding branch. Points A and B therefore correspond to two fundamentally different modes of the same algorithm: “ordinary continuation” and “affine correction.”



4.3. FIG. 2B — Result of the New Method

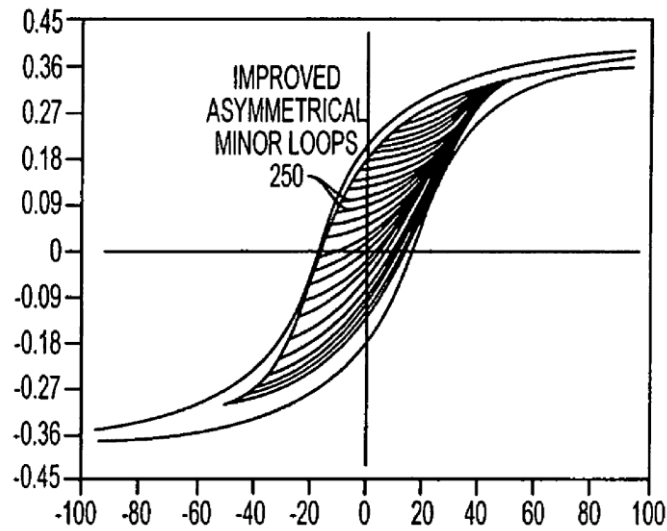


FIG. 2B
NEW METHOD

Patent FIG. 2B: improved asymmetrical minor loops 250 constructed by the new method.

FIG. 2B demonstrates the objective of the patent: the family of asymmetrical minor loops 250 remains inside the major loop. This does not mean that the model reproduces the full microscopic physics of the domain structure. Its purpose is to preserve a physically plausible $B(H)$ geometry in a computationally lightweight circuit model.

4.4. FIG. 3A — Algorithm Branch for Increasing H

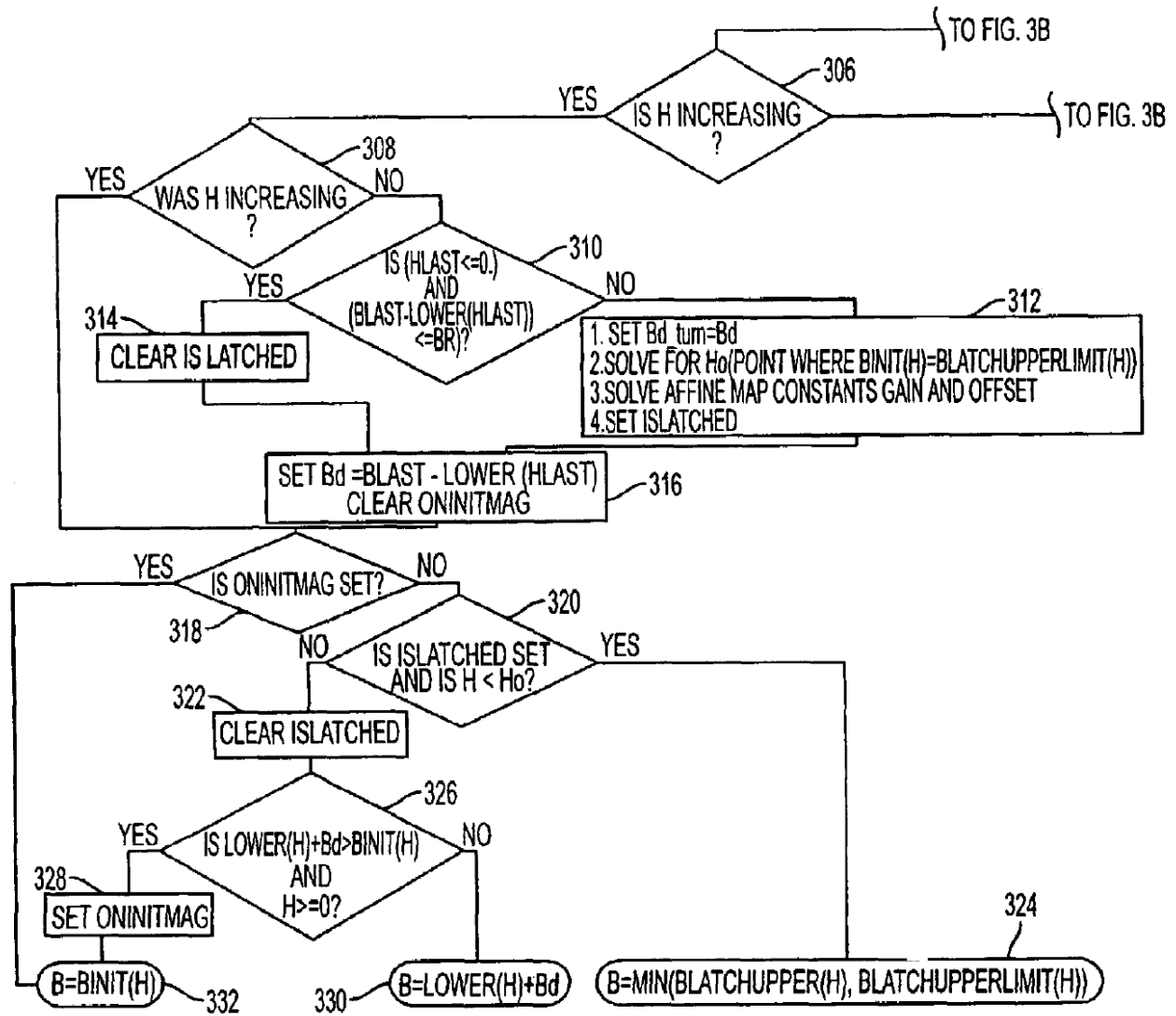


FIG. 3A

Patent FIG. 3A: algorithm branch for currently increasing H, steps 306-332.

FIG. 3A implements three levels of decision making. First, it determines whether a reversal has occurred (step 308). If there was a reversal, it then checks whether an ordinary symmetrical branch is possible (step 310); if not, an affine map is constructed (step 312). Finally, steps 318-332 determine which expression should be used at the current H: Binit, the latched affine branch, or the ordinary Lower(H)+Bd branch.

4.5. FIG. 3B — Algorithm Branch for Decreasing H

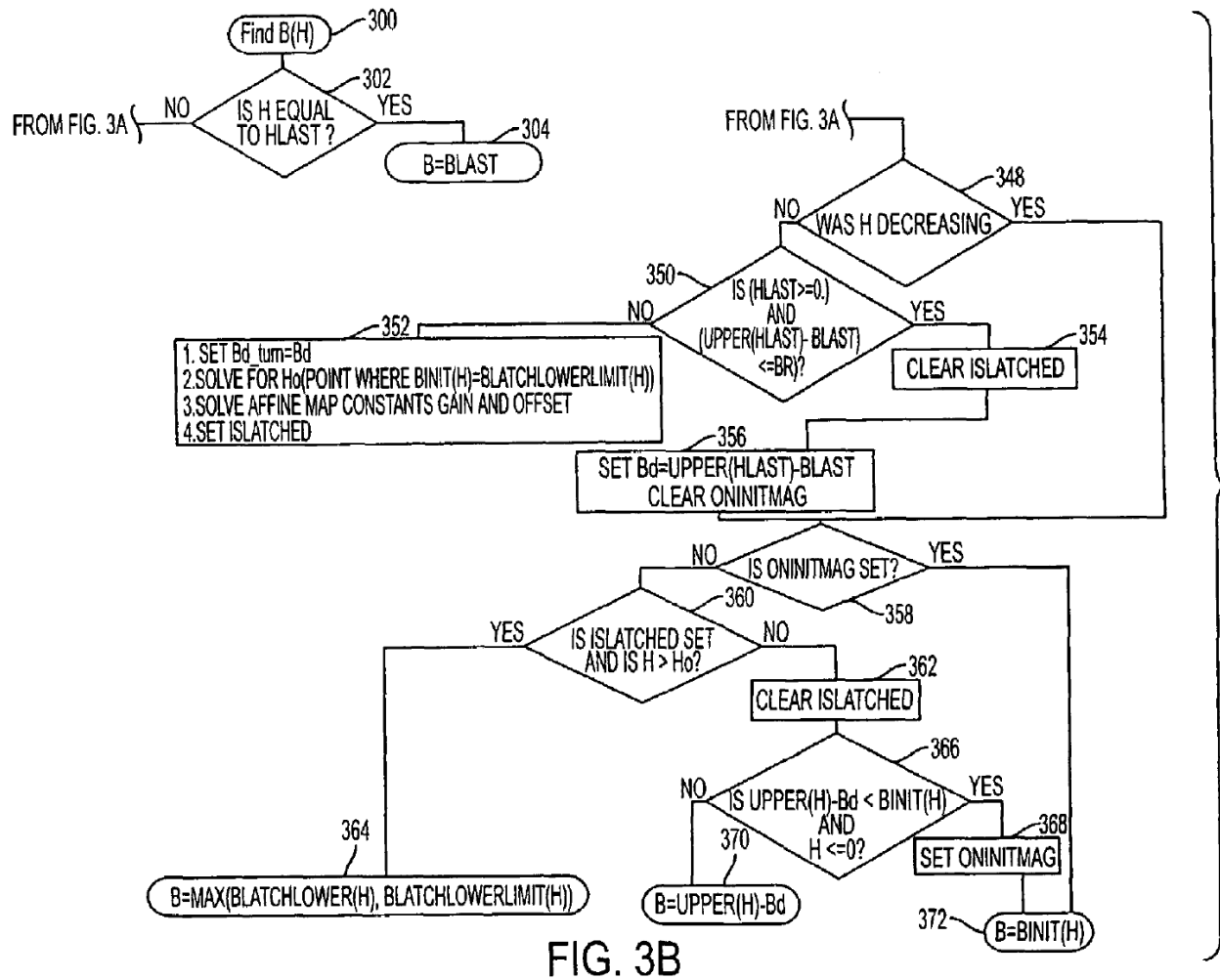
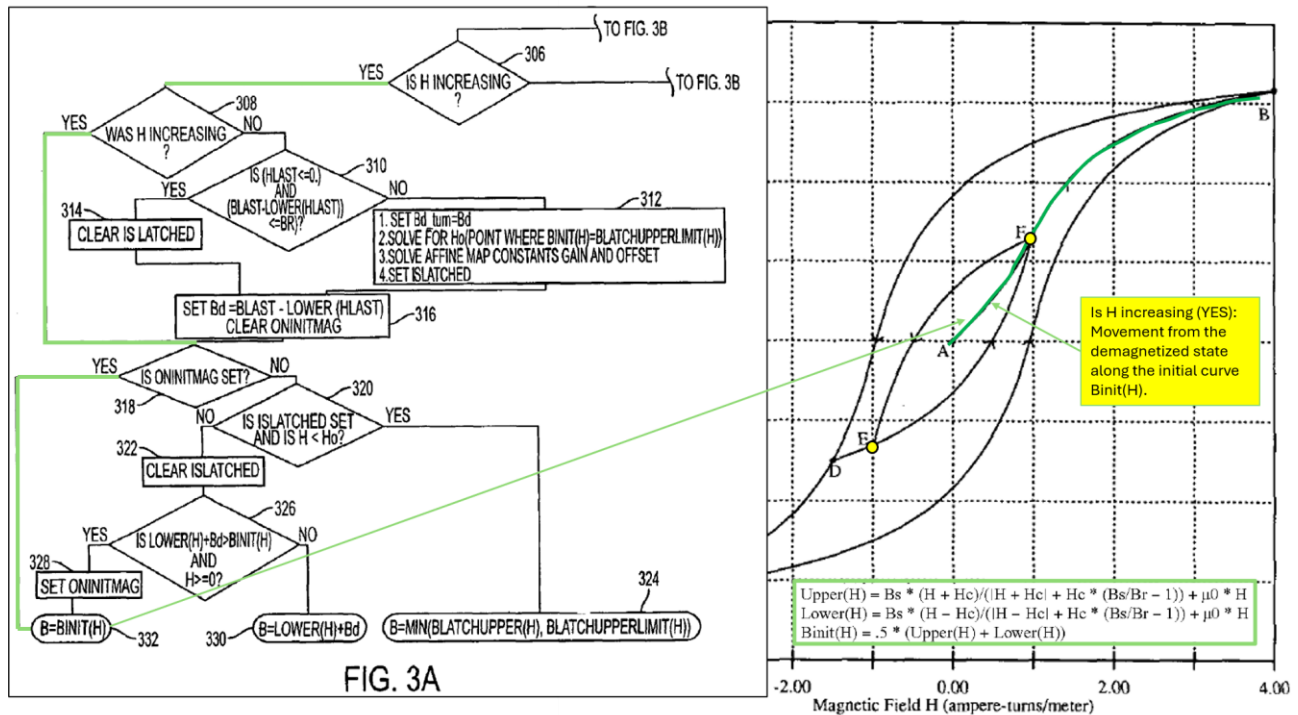


FIG. 3B

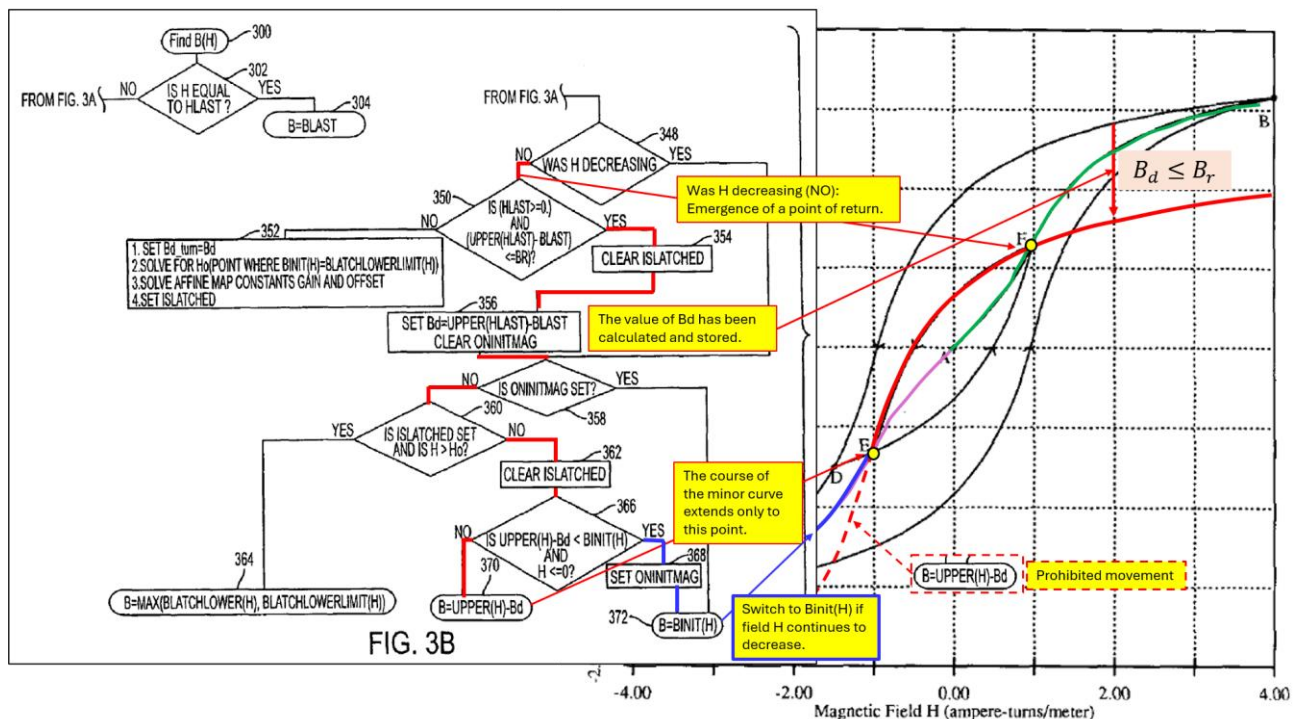
Patent FIG. 3B: general H = Hlast check and the mirrored branch for decreasing H, steps 300-304 and 348-372.

The upper part of FIG. 3B contains the general steps 300-304: when $H = H_{last}$, Blast is returned. The main part, steps 348-372, is the mirrored counterpart of FIG. 3A. For decreasing field, $Upper(H) - B_d$, BlatchLower, and BlatchLowerLimit are used, while the condition for entering the initial magnetization path is checked in the negative-H region.

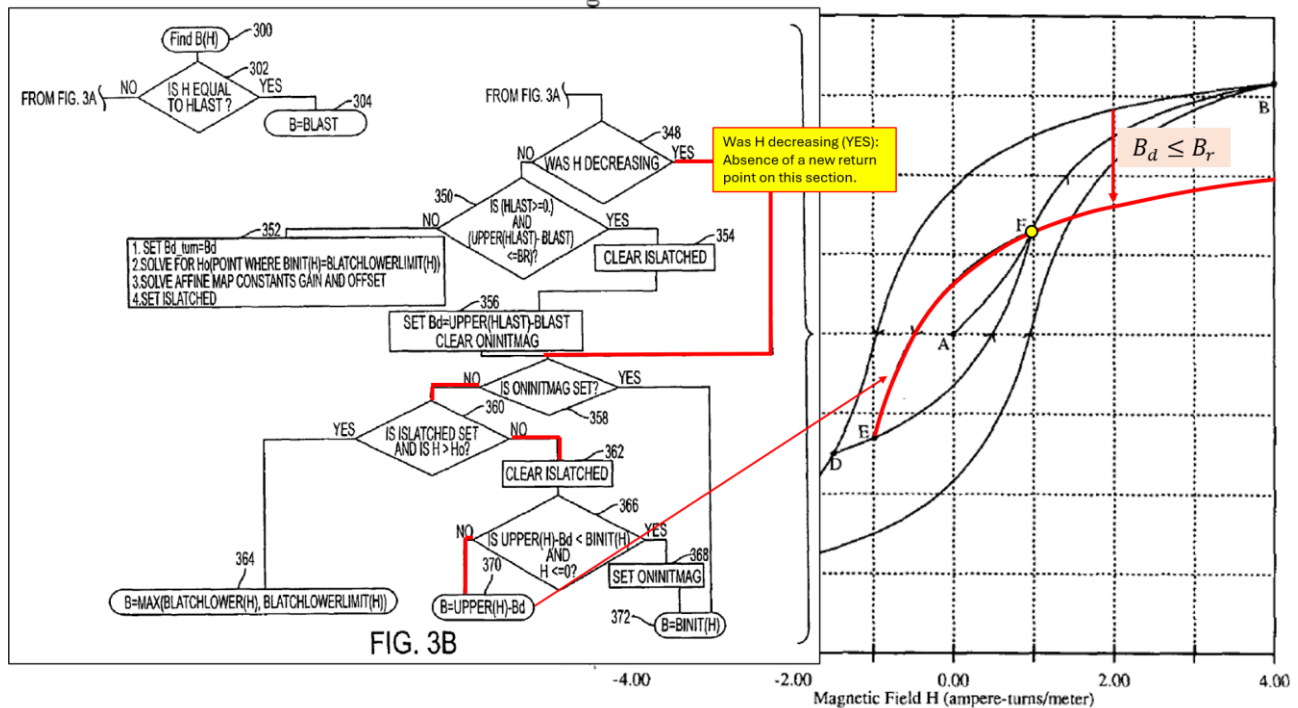
Below we show several constructions illustrating how to follow the patent algorithm diagrams when building ordinary symmetrical loops; in this part they reproduce the Chan algorithm. Because we are interested in modelling the magnetic-memory effect of the material, the process starts from the fully demagnetized state A and proceeds to some point F on the initial magnetization curve.



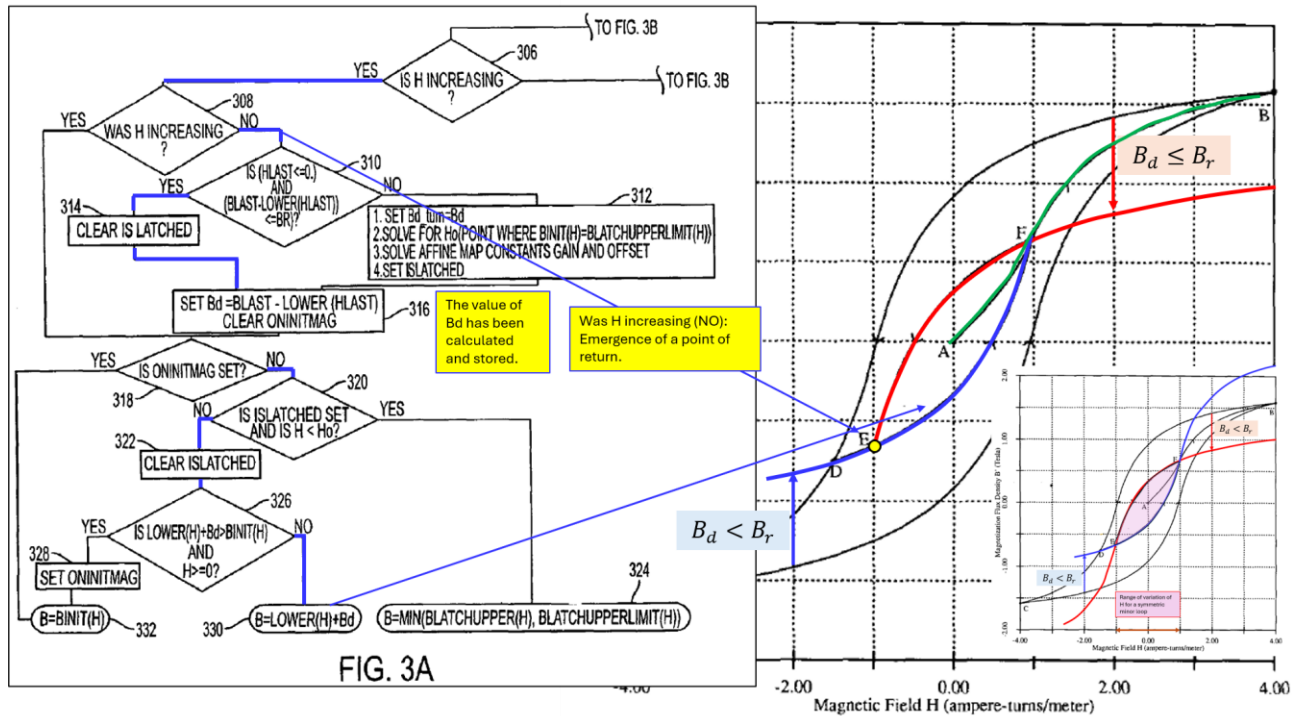
After the reversal at point F, motion continues along the minor curve (red), obtained by shifting the original upper major-loop branch downward. After this minor curve intersects Binit(H) at point E, the motion continues along Binit(H) (blue curve).



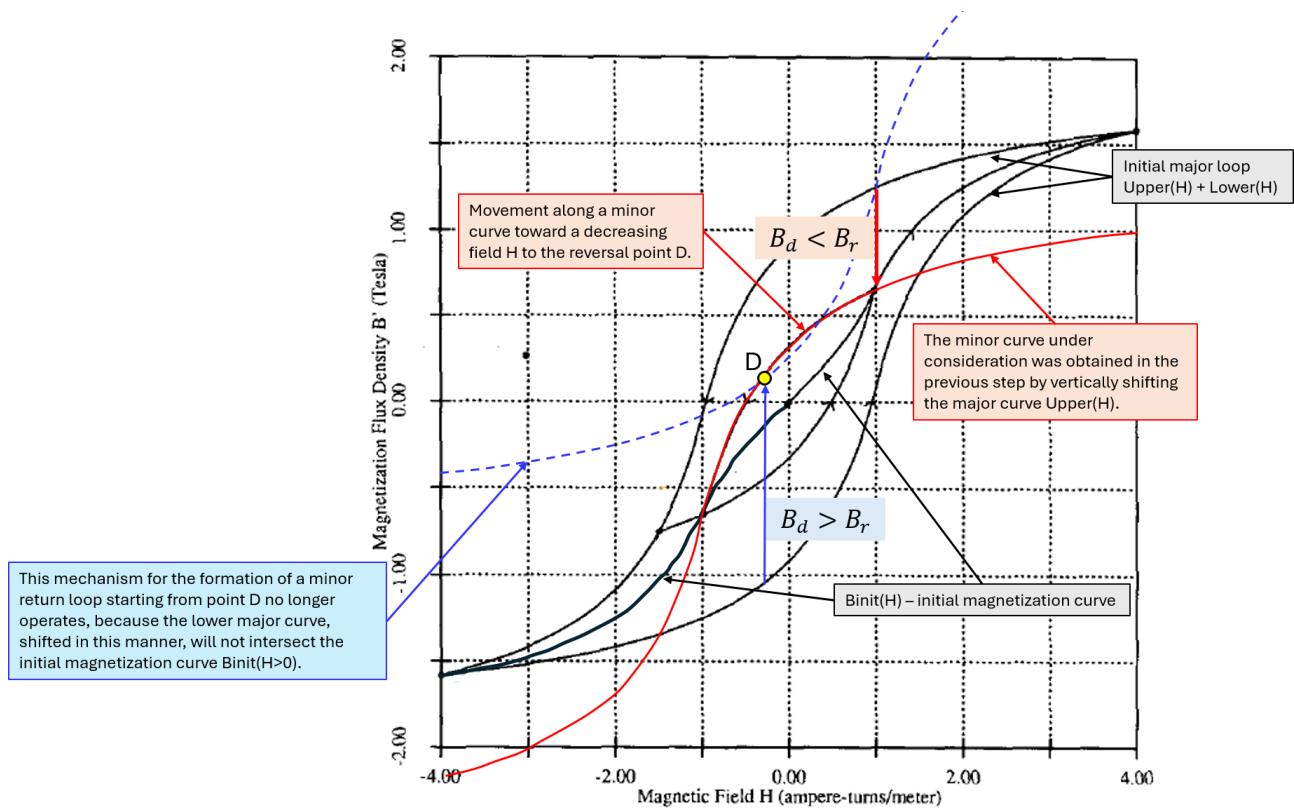
Motion along the minor curve with decreasing field without forming a new return point.



After the reversal at point E, motion continues along the minor curve (blue), obtained by shifting the original lower major-loop branch upward. After this minor curve intersects Binit(H) at point F, motion continues along Binit(H) (green curve). Since both return points E and F lie symmetrically on the initial curve Binit(H), a closed symmetrical minor loop is formed (shown in pink in the inset to the figure).



Now consider the mechanism for forming asymmetrical minor loops when a simple vertical shift of the major-loop curves is no longer possible because the condition $B_d \leq B_r$ is violated, as shown in the figure below. First, we move along a certain minor curve to the return point D. However, the return curve from D can no longer be obtained as $\text{Lower}(H) + B_d$ (a vertical shift of the major-loop curve), because such a shift would exceed B_r : the resulting minor curve (blue dashed line) would not intersect the initial magnetization curve B_{init} , which is not physically admissible.



The method for forming an asymmetrical minor loop in the Chan algorithm, shown below, initially appears rather nonphysical because it extends outside the major loop. As demonstrated in the patent, when modelling an inductor with a ferromagnetic core, the Chan model can produce a current response that is not observed physically under certain excitation conditions.

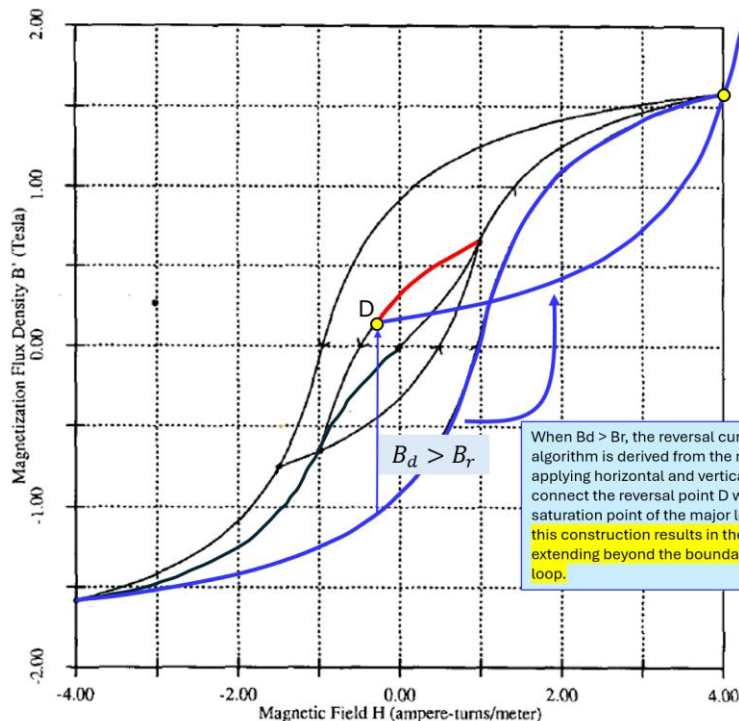


FIG. 1 illustrates the offending asymmetrical minor hysteresis loops 105 present in Chan's model. Asymmetric hysteresis loops are of two forms. Using Chan's notation of B_d as the constant of vertical translation between a minor loop branch and the corresponding branch of the major loop, if there is a minor loop with B_d less than B_r that can be extended beyond the initial magnetization path to meet the turning point, then that is used for that branch of the asymmetric hysteresis loop. But if there is no such minor loop, then Chan's technique translates the lower major hysteresis loop horizontally and vertically to fit the turning point. This translation results in the offending asymmetrical minor hysteresis loops 105, which exhibit non-physical behavior in that they extend beyond the major hysteresis loop 110. The inductor appears to be saturated when traversing the offending asymmetrical minor hysteresis loops 105 because of the flatness of the slope of these loops. As one skilled in the art will appreciate, circuit simulations or other uses of Chan's model tend to produce erroneous results because the inductor appears to be saturated when it should in fact not be saturated.

Explanation from the patent

The patent proposes a different method for forming asymmetrical minor loops, discussed below, which is free from the shortcomings of the Chan algorithm. Depending on which minor curve the trajectory follows before reaching the anomalous return point:

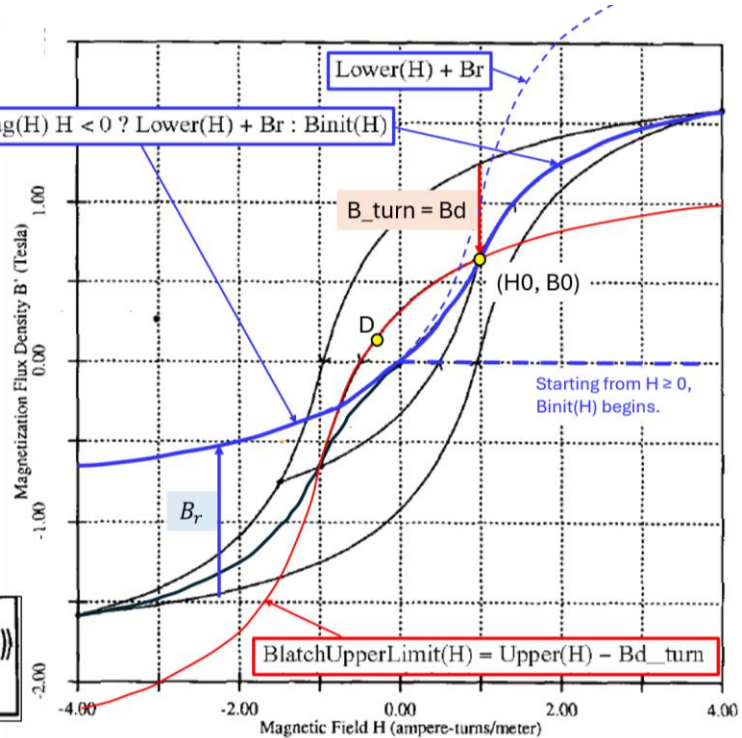
- The coordinates of the intersection point (H_0, B_0) between the $\text{BlatchUpperLimit}(H)$ or $\text{BlatchLowerLimit}(H)$ curve and the magnetization curve B_{init} are found.
- An auxiliary curve $\text{UpperBmag}(H)$ or $\text{LowerBmag}(H)$ is constructed.
- The curve $\text{UpperBmag}(H)$ or $\text{LowerBmag}(H)$ is subjected to an affine transformation that maps it onto points D and (H_0, B_0) . This curve forms the return path up to point (H_0, B_0) , after which the motion continues along the magnetization curve $B_{\text{init}}(H)$. At the joining point (H_0, B_0) , the derivative may be discontinuous, but this is not important because the numerical calculations do not use the derivative dB/dH .

The auxiliary curve $\text{UpperBmag}(H)$ is smoothly constructed by joining two segments:
 $\text{Lower}(H) + Br$ for $H < 0$
 $B_{\text{init}}(H)$ for $H \geq 0$

Steps 1–4 of the patent algorithm :

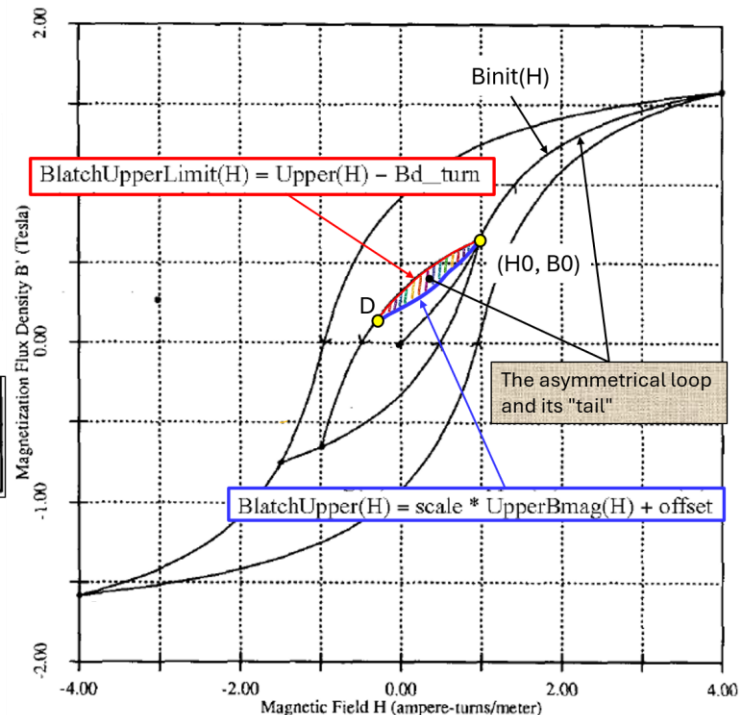
1. Set $B_{\text{d_turn}} = B_{\text{d}}$, where B_{d} is known from the previous step.
2. Solve the nonlinear equation to find the coordinates of the point (H_0, B_0) where $B_{\text{init}}(H)$ intersects either $\text{BlatchUpperLimit}(H) = \text{Upper}(H) - B_{\text{d_turn}}$ or $\text{BlatchLowerLimit}(H) = \text{Lower}(H) + B_{\text{d_turn}}$ (depending on the minor curve).
3. Construct an auxiliary curve, $\text{UpperBmag}(H)$ or $\text{LowerBmag}(H)$, and then align it with points D and (H_0, B_0) using an affine transformation. The resulting transformed curve forms the return path to (H_0, B_0) . Subsequently, the process continues along the initial magnetization curve $B_{\text{init}}(H)$.
4. Set the **ISLATCHED** flag to **True** while movement occurs between D and (H_0, B_0) . Otherwise, set **ISLATCHED** to **False**.

1.SET $B_{\text{d_turn}}=B_{\text{d}}$
 2.SOLVE FOR H_0 (POINT WHERE $B_{\text{INIT}}(H)=\text{BLATCHUPPERLIMIT}(H)$)
 3.SOLVE AFFINE MAP CONSTANTS GAIN AND OFFSET
 4.SET ISLATCHED



$\text{Upper}(H) = B_s * (H + H_c) / (|H + H_c| + H_c * (B_s / B_r - 1)) + \mu_0 * H$
 $\text{Lower}(H) = B_s * (H - H_c) / (|H - H_c| + H_c * (B_s / B_r - 1)) + \mu_0 * H$
 $B_{\text{init}}(H) = .5 * (\text{Upper}(H) + \text{Lower}(H))$
 $\text{UpperBmag}(H) \ H < 0 ? \text{Lower}(H) + Br : B_{\text{init}}(H)$
 $\text{LowerBmag}(H) \ H > 0 ? \text{Upper}(H) - Br : B_{\text{init}}(H)$
 $\text{BlatchUpper}(H) = \text{scale} * \text{UpperBmag}(H) + \text{offset}$
 $\text{BlatchLower}(H) = \text{scale} * \text{LowerBmag}(H) + \text{offset}$
 $\text{BlatchUpperLimit}(H) = \text{Upper}(H) - B_{\text{d_turn}}$
 $\text{BlatchLowerLimit}(H) = \text{Lower}(H) + B_{\text{d_turn}}$

1.SET $B_{\text{d_turn}}=B_{\text{d}}$
 2.SOLVE FOR H_0 (POINT WHERE $B_{\text{INIT}}(H)=\text{BLATCHUPPERLIMIT}(H)$)
 3.SOLVE AFFINE MAP CONSTANTS GAIN AND OFFSET
 4.SET ISLATCHED



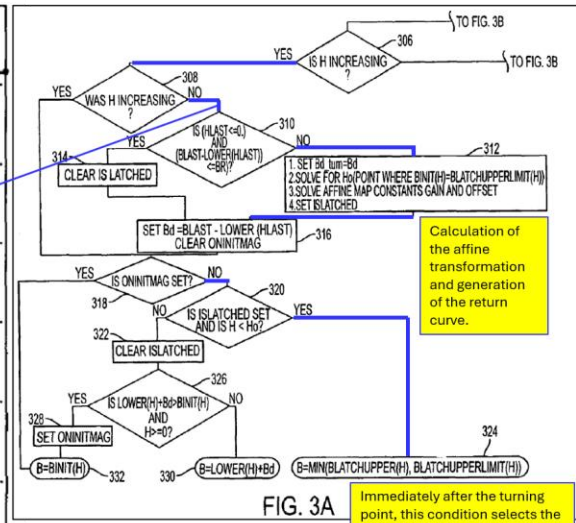
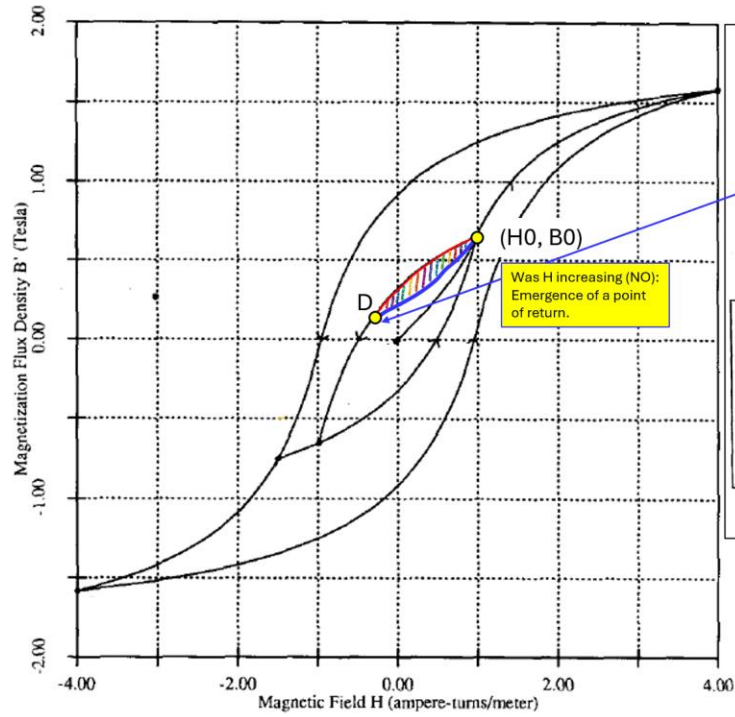


FIG. 3A

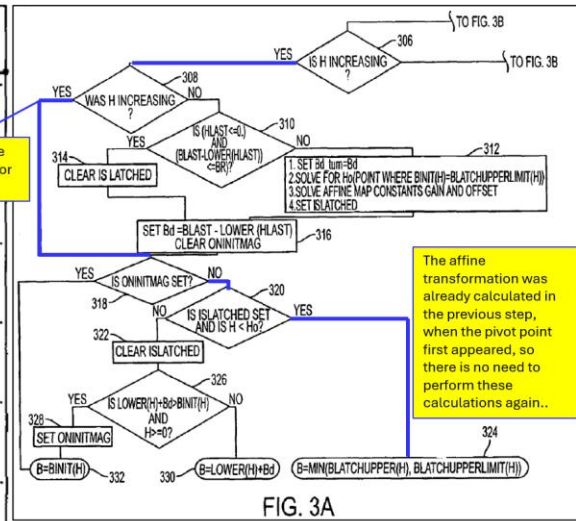
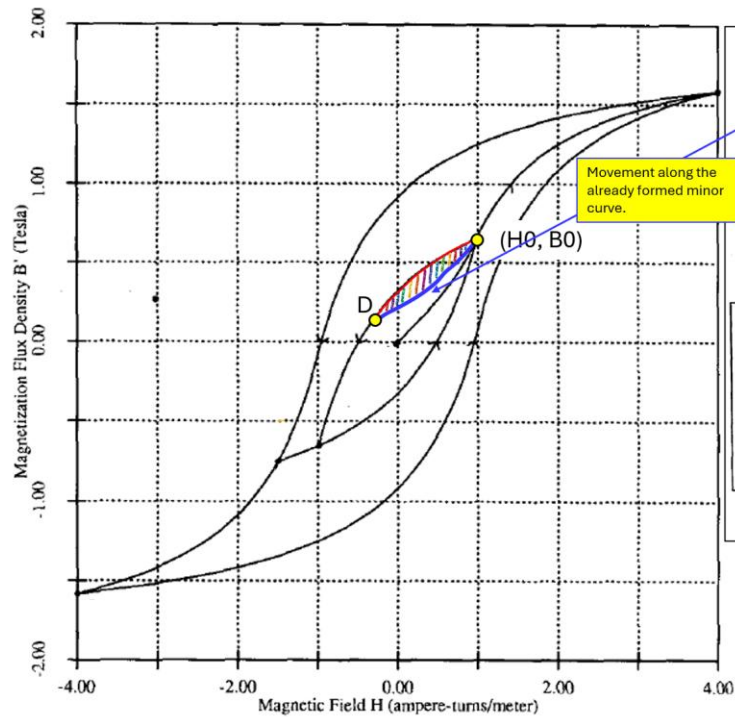
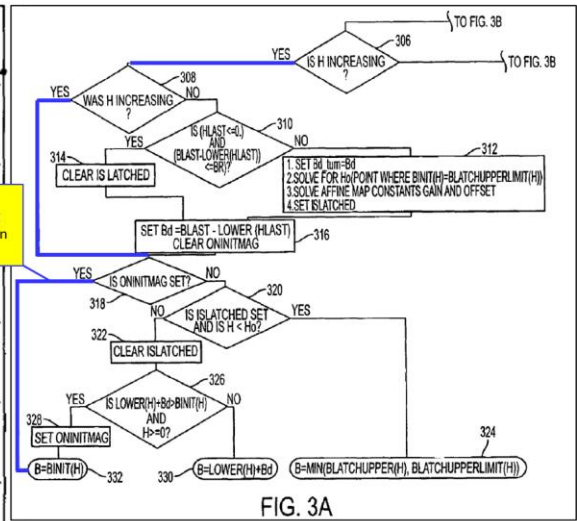
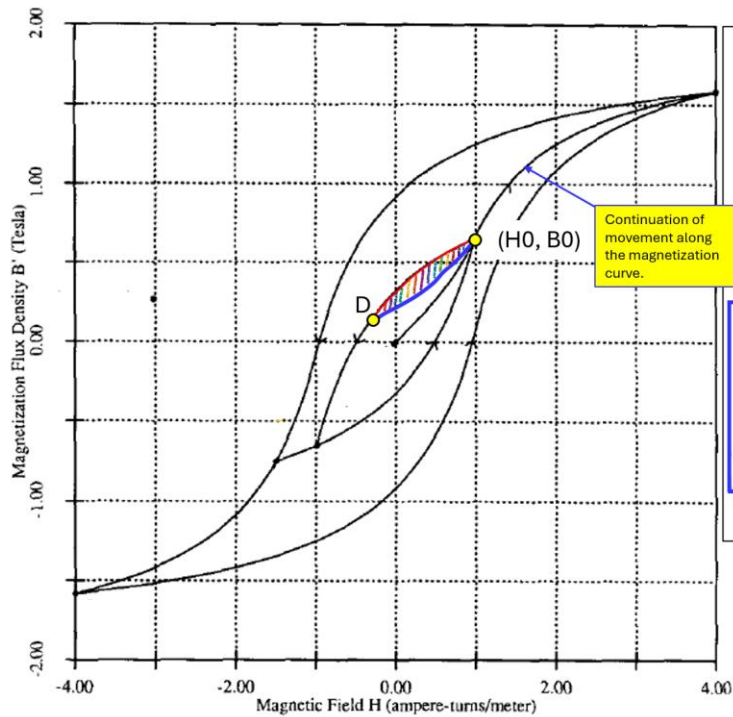
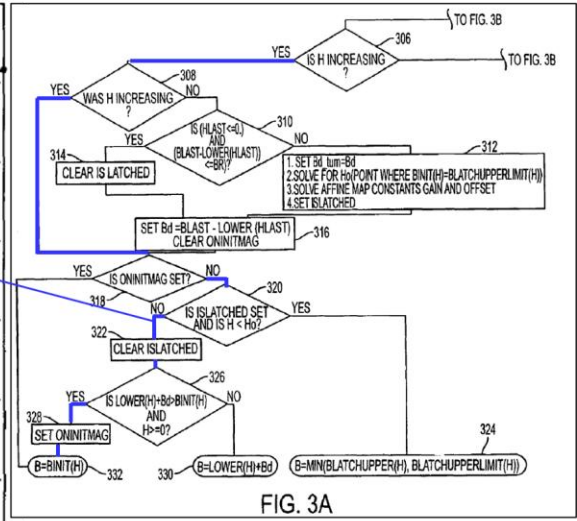
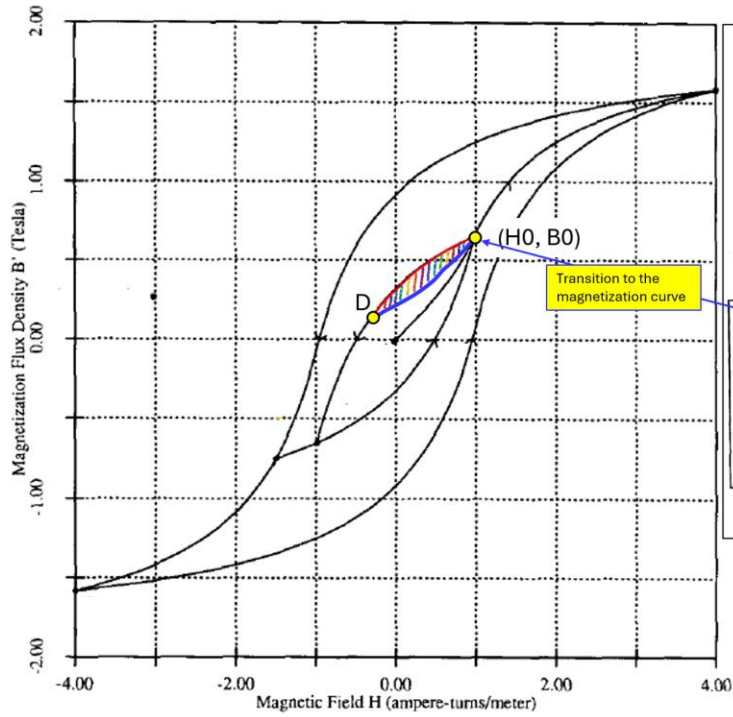


FIG. 3A



The modified Chan algorithm from the patent is implemented in the widely used [LTspice](#) electronic simulator from [Analog Devices](#) (formerly [Linear Technology](#)). We replicated it in Python for future use in the magnetic rapper project. Python algorithms are discussed below.

4.6. FIG. 4 — Circuit Simulation Example

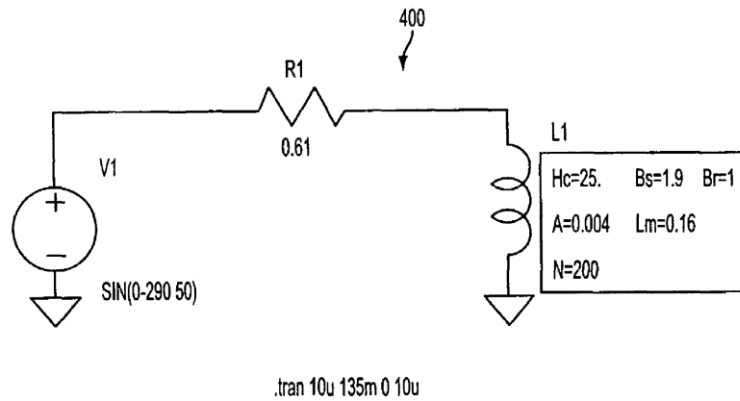


FIG. 4

Patent FIG. 4: test SPICE circuit with a source, $R = 0.61 \Omega$, and a nonlinear inductor.

FIG. 4 shows not a separate magnetostatic test, but the inclusion of the model in a dynamic circuit simulation in LTspice. The sinusoidal source is specified as $\text{SIN}(0 \text{ } -290 \text{ } 50)$, with resistor $R1 = 0.61 \Omega$ and inductor $L1$ connected in series. In the example, the core parameters are $H_c = 25 \text{ A/m}$, $B_s = 1.9 \text{ T}$, $B_r = 1 \text{ T}$, $A = 0.004 \text{ m}^2$ (core cross-sectional area), $L_m = 0.16 \text{ m}$ (mean magnetic path length of the core), and $N = 200$ (number of turns of the inductor coil). The core contains no gap, but a gap can be introduced if necessary using the parameter L_g (length). When a gap is present, L_m includes only the magnetic portion of the path, so the total core path length is $L_m + L_g$. The core is assumed to have a uniform cross-section along its length.

4.7. FIG. 5A and FIG. 5B — Effect of the Model on Inductor Current

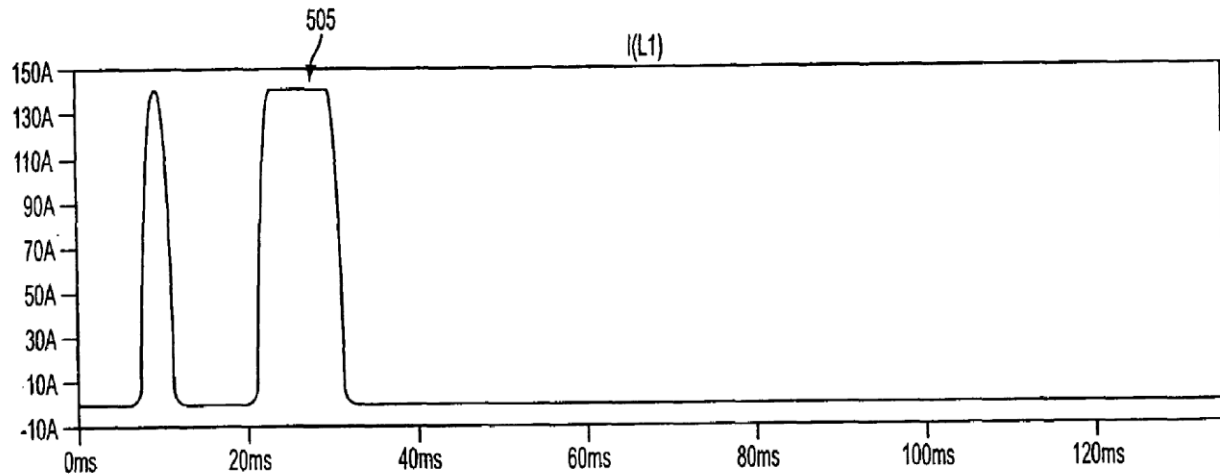


FIG. 5A

Patent FIG. 5A: current $I(L1)$ with the prior-art model; region 505 corresponds to false excessive saturation.

In FIG. 5A the current reaches very large values in region 505 because the prior-art asymmetrical branch has too small a slope and the model interprets the core as saturated.

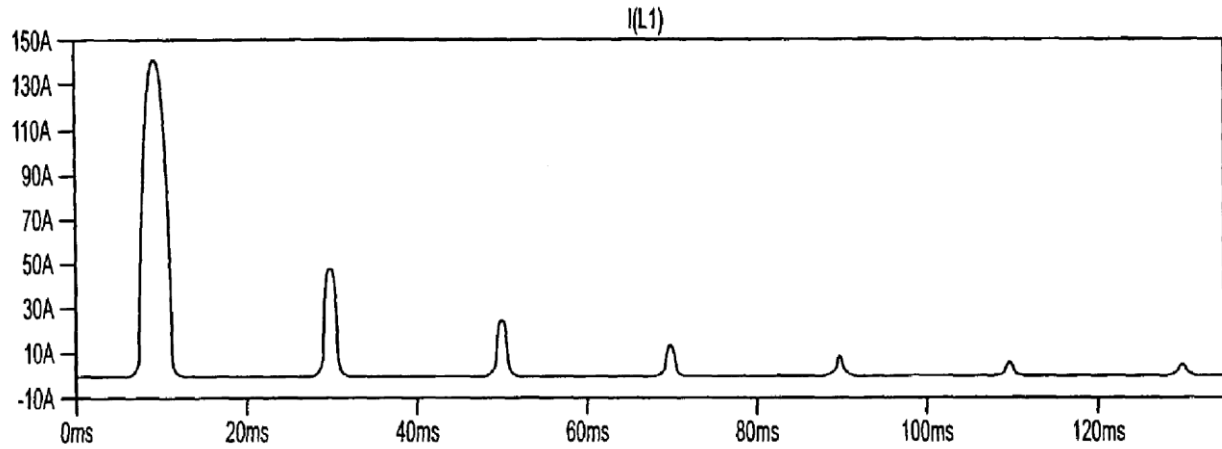


FIG. 5B

Patent FIG. 5B: current $I(L1)$ with the new Engelhardt method.

FIG. 5B shows that, after correcting the asymmetrical branches, this effect is substantially suppressed. For the patent, this is a practical demonstration that a geometric defect in $B(H)$ can significantly distort the electrical simulation result.

4.8. FIG. 6 — Computational Implementation

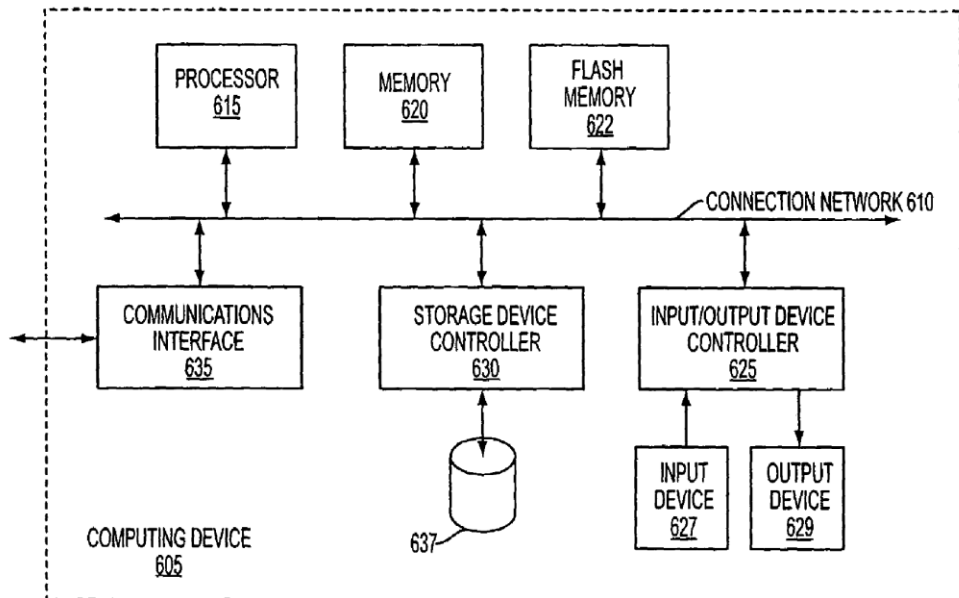


FIG. 6

Patent FIG. 6: generalized computing system for executing the algorithm.

FIG. 6 contains no hysteresis-specific physics; it shows a conventional computing architecture: processor, memory, flash memory, I/O controller, storage controller, communications interface, etc. Its role in the patent is to establish that the algorithm can be implemented in software, hardware, or a combination of the two.

5. Complete FIG. 3 Algorithm: Steps 300-372

5.1. General Steps 300-306

Step	Operation	Meaning
300	Find B(H)	Algorithm entry: determine B for a new value of H while preserving the stored history.
302	Is H equal to Hlast?	If the field has not changed, there is no reason to recalculate the branch or state.
304	B = Blast	The previous value of B is returned; the history is unchanged.
306	Is H increasing?	Select one of the two mirrored halves: FIG. 3A for $H \uparrow$ or FIG. 3B for $H \downarrow$.

5.2. Increasing H: Steps 308-316 — Reversal Handling

Step 308 asks whether H was also increasing on the previous step. If so, the current segment continues without a new reversal. If not, the point (Hlast, Blast) is a turning point and a new branch must be determined:

Step 310: $(Hlast \leq 0)$ AND $(Blast - Lower(Hlast) \leq Br)$?

The quantity $Blast - Lower(Hlast)$ is precisely the new Bd that will be assigned at step 316. If the condition is true, a symmetrical minor branch is admissible, so step 314 simply clears IsLatched. If the condition is false, step 312 activates the special correction.

5.3. Step 312 — Construction of the Asymmetrical Branch for $H \uparrow$

1. SET $Bd_turn = Bd$. The Bd of the old branch that existed before the reversal is stored.
2. SOLVE FOR $H0$. The nonlinear equation $Binit(H0) = BlatchUpperLimit(H0) = Upper(H0) - Bd_turn$ is solved.
3. SOLVE AFFINE MAP CONSTANTS GAIN AND OFFSET. The scale and offset for $BlatchUpper$ are determined.
4. SET IsLatched. The flag tells the subsequent steps that the affine branch should be used up to $H0$.

Step 316 is then executed regardless of whether step 312 or 314 was selected:

$Bd = Blast - Lower(Hlast); \text{ OnInitMag} = \text{False}$

The order of operations is essential: Bd_turn must receive the old Bd before Bd is redefined for the new increasing branch.

5.4. Steps 318-332 — Formula Selection for $H \uparrow$

Step 318 first checks OnInitMag. If it is set, step 332 immediately gives $B = Binit(H)$. Otherwise, the latch is considered:

Step 320: $IsLatched$ AND $H < H0$?

If yes, step 324 calculates:

$B = \text{MIN}[BlatchUpper(H), BlatchUpperLimit(H)]$

MIN is a protective operation: even if the affine mapping itself tends to cross the old loop, the result remains no higher than $Upper(H) - Bd_turn$.

If the latch is inactive or H has reached/passed $H0$, step 322 clears IsLatched. Step 326 then checks whether the ordinary branch $Lower(H) + Bd$ has crossed the initial curve for $H \geq 0$:

$Lower(H) + Bd > Binit(H)$ AND $H \geq 0$?

If the condition is true, step 328 sets OnInitMag and step 332 returns $Binit(H)$. Otherwise, step 330 returns the ordinary minor branch $Lower(H) + Bd$.

5.5. Decreasing H: Steps 348-356 — Mirrored Reversal

Step 348 asks whether H was decreasing on the previous step. If not, a reversal from increasing to decreasing has occurred:

Step 350: $(H_{last} \geq 0)$ AND $(Upper(H_{last}) - Blast \leq Br)$?

If the condition is true, affine correction is not required and step 354 clears IsLatched. If it is false, step 352 constructs the mirrored affine solution: $Bd_turn = Bd$, H_0 is determined from the intersection of Binit with BlatchLowerLimit, then gain/offset are calculated and IsLatched is set:

Step 356: $Bd = Upper(H_{last}) - Blast$; OnInitMag = False

5.6. Steps 358-372 — Formula Selection for H↓

Step 360: IsLatched AND $H > H_0$?

With an active latch, step 364 is used:

$B = \text{MAX}[BlatchLower(H), BlatchLowerLimit(H)]$

MAX is the mirrored limiter: the new branch cannot fall below $Lower(H) + Bd_turn$.

After the latch is cleared, step 366 checks for return to the initial magnetization path:

$Upper(H) - Bd < Binit(H)$ AND $H \leq 0$?

If the condition is true, step 368 sets OnInitMag and step 372 returns Binit(H). Otherwise, step 370 gives $B = Upper(H) - Bd$.

The algorithm is easier to understand not as a collection of unrelated if-statements, but as a small state machine. In it, the geometric branch is determined not only by the current H but also by the history.

Variable	Role
Hlast, Blast	Last calculated point of the trajectory. Required for detecting a reversal and ensuring continuity.
dir / previous_direction	Direction in which H was changing before the current step.
OnInitMag	The trajectory is on Binit(H).
IsLatched	The special affine branch is active up to the point H_0 .
Bd	Vertical shift of the current ordinary minor branch.
Bd_turn	Frozen Bd of the preceding branch at the turning point; used as a boundary limit.
H_0	Field value at which the latched mode ends.
scale, offset	Parameters of the affine transformation of the basis function.

Thus, $B = B(H)$ here is not a single-valued function of the current H alone. It is more accurate to write $B = B(H, \text{state})$, where state stores the memory of the trajectory. This is the computational form of hysteresis.

6. Patent Claims

Claims 1-9 are organized into three parallel groups. The following is a technical rather than legal summary.

Claim	Type	Content
1	Computer-implemented method	Upper/Lower major branches; Binit as their average; symmetrical minor loops as vertical shifts; asymmetrical loops as continuation of a symmetrical branch or, when that is impossible, as a bounded function based on a basis function; application in circuit simulation and output of the result.
2	Refinement of claim 1	The problematic asymmetrical branch is an affine map of the function $B_{basis}(H)$, matched to the turning point and to the intersection point of the minor branch with the initial magnetization path.
3	Refinement of claim 1	The model can be applied to a gapped core.
4-6	System claims	The same ideas are formulated as a computing system/processor; claim 5 — affine map, claim 6 — gap.
7-9	Computer-readable medium	The same ideas are formulated as program code on a computer-readable medium; claim 8 — affine map, claim 9 — gap.

From an engineering standpoint, the most important distinguishing feature is not the major-loop formula itself, but the method for constructing a problematic asymmetrical minor branch as a bounded affine map of a basis function.

7. Python Project for Hysteresis-Loop Calculation

The Engelhardt patent can be regarded as a minimal stateful hysteresis model built around a simple analytical major loop. Ordinary minor branches are implemented as vertical shifts of Upper/Lower. The novelty and practical value appear only at those reversals where an ordinary symmetrical branch cannot be continued to the turning point; in that case, a bounded affine map of a composite basis function is created. [The current Python project](#) [4] reproduces this structure quite closely to the original flowchart. `BH_increasing()` and `BH_decreasing()` are almost mirrored transcriptions of FIG. 3A/3B; `H0` is solved numerically; scale and offset are calculated from endpoint constraints; `main.py` stores the direction and state history; `Hscan.py` provides the excitation trajectories. The next qualitative step is not to add new formulas, but to automate invariant verification and create diagnostic tests that repeatedly force the model into steps 312 and 352.

After that, the model can be used as a stand-alone material model for calculating $B(H)$, and then incorporated into a broader dynamic calculation of a magnetic circuit: flux linkage, back EMF, excitation current, force, and mechanical motion of the actuator.

7.1 Project Structure

The project is stored in the folder “Chan-Engelhardt-Model-Symmetrical&Asymmetrical-BHloops” in the [GitHub repository](#). [4]

```
project/
├── main.py
├── BH.py
├── Hscan.py
├── Parameters.py
└── Parameters.txt
```

Responsibilities are distributed as follows:

- `Parameters.py` — loads numerical parameters by name from the text file `Parameters.txt`;
- `BH.py` — contains all physical and algorithmic $B(H)$ logic: the major loop Upper&Lower, initial magnetizing Binit, basis functions, H_0 , affine map, and increasing/decreasing branches `BH_increasing/BH_decreasing`;
- `Hscan.py` — generates the excitation-field history $H(t)$, including test modes;
- `main.py` — performs the discrete pass through H values, stores state, calls `BH_increasing/BH_decreasing`, accumulates B values, and plots $B(H)$.

A concise map of functions and variables:

Element	File	Purpose
Upper(H)	BH.py	Upper major-loop branch.
Lower(H)	BH.py	Lower major-loop branch.
Binit(H)	BH.py	Initial magnetization path.
UpperBmag(H)	BH.py	Basis function for the upper affine map.
LowerBmag(H)	BH.py	Basis function for the lower affine map.
BlatchUpper/Lower	BH.py	Affinely transformed basis functions.
BlatchUpperLimit/LowerLimit	BH.py	Limits that prevent crossing of the preceding loop.
solve_H0_upper/lower	BH.py	Bracketing + bisection for H0.
solve_affine_upper/lower	BH.py	Calculation of scale/offset from two endpoint conditions.
BH_increasing	BH.py	FIG. 3A, current H increasing.
BH_decreasing	BH.py	FIG. 3B, current H decreasing.
Hscan1	Hscan.py	Exponentially decaying 60 Hz sinusoid.
Hscan2	Hscan.py	Asymmetrical/nested stress scan around a positive bias.
parameters	Parameters.py	Dictionary of parameters read from Parameters.txt.
previous_direction	main.py	Python analogue of the patent variable dir.

7.2 BH.py Function Module

At the beginning, BH.py imports the parameters dictionary and fixes Bs, Br, and Hc once as module-level variables. This means that Parameters.txt is read only when Parameters.py is imported, and calls to Upper(), Lower(), and the other functions do not access the disk:

```
from Parameters import parameters as par

pi = 3.1415926535897932384626433832795
mu0 = 4 * pi * 1.0e-7

Bs = par["Bs"]
Br = par["Br"]
Hc = par["Hc"]
```

The major-loop and initial-magnetization functions literally reproduce Table 1 of the patent:

```
def Binit(H):
    return 0.5 * (Upper(H) + Lower(H))

def UpperBmag(H):
    return Lower(H) + Br if H < 0.0 else Binit(H)

def LowerBmag(H):
    return Upper(H) - Br if H > 0.0 else Binit(H)
```

Similarly, BlatchUpper/BlatchLower implement the affine map, while BlatchUpperLimit/BlatchLowerLimit define the limits required by steps 324 and 364.

The patent requires “solve for H_0 ” but does not prescribe a numerical method. BH.py uses a two-stage solution: first the root is bracketed by expanding the interval, and then it is refined by bisection. For the increasing branch, the search proceeds to the right from Hlast; for the decreasing branch, to the left. This is a reasonable and robust choice because the functions involved are continuous over the region of interest.

```
def f(H):
    return Binit(H) - BlatchUpperLimit(H, Bd_turn)

# search a bracket to the right of Hlast
# then use bisection until |f(H)| < tol
```

The value $\text{tol} = 1\text{e-}10$ refers to the residual in B, while $\text{max_iter} = 100$ limits both the bracket search and the bisection loop.

The functions `solve_affine_upper()` and `solve_affine_lower()` perform a two-point fit. For the upper function, the code forms $x_1 = \text{UpperBmag}(\text{Hlast})$, $y_1 = \text{Blast}$, $x_2 = \text{UpperBmag}(H_0)$, $y_2 = \text{Binit}(H_0)$, and then calculates the slope and intercept. The denominator check protects against the degenerate case in which the two x-coordinates of the basis function coincide.

The signature of `BH_increasing()` corresponds to step 308, “WAS H INCREASING?”:

```
def BH_increasing(
    H, was_inc, Hlast, Blast, OnInitMag, IsLatched,
    Bd, Bd_turn, H0, scale, offset
):
```

It receives the current H and the complete set of state variables associated with the previous point. Inside the function, variables with the suffix `*_new` are first created equal to the old values. This allows only those state variables that are actually changed by the corresponding block of FIG. 3A to be modified.

Patent step	Python	Meaning
308	if not was_inc	Reversal from decreasing to increasing.
310	Hlast <= 0 and Blast - Lower(Hlast) <= Br	Checks whether an ordinary symmetrical branch is sufficient.
314	IsLatched_new = False	Affine correction is not required.
312.1	Bd_turn_new = Bd	Stores the old Bd.
312.2	solve_H0_upper(...)	Finds the intersection of Binit with Upper(H)–Bd_turn.
312.3	solve_affine_upper(...)	Calculates scale and offset.
312.4	IsLatched_new = True	Activates the latch.
316	Bd_new = Blast - Lower(Hlast)	Parameter of the new increasing branch; OnInitMag=False.
318	if OnInitMag_new	If on the initial magnetization path, B=Binit.
320	IsLatched_new and H < H0_new	Checks whether the affine segment is still active.
324	min(BlatchUpper, BlatchUpperLimit)	Bounded affine branch.
322	IsLatched_new = False	Latch cleared.
326	Lower(H)+Bd_new > Binit(H) and H>=0	Checks entry onto the initial path.
328/332	OnInitMag=True; B=Binit(H)	Transition to Binit.
330	B=Lower(H)+Bd_new	Ordinary increasing minor branch.

The function always returns the same tuple of eight elements: B and seven updated state parameters. Keeping this order fixed is important because `main.py` unpacks the result positionally.

The function BH_decreasing() is deliberately made as a mirror image of BH_increasing(), corresponding to step 348, “WAS H DECREASING?”

Patent step	Python	Meaning
348	if not was_dec	Reversal from increasing to decreasing.
350	Hlast >= 0 and Upper(Hlast) - Blast <= Br	Checks whether a symmetrical branch is admissible.
354	IsLatched_new = False	Affine correction is not required.
352.1	Bd_turn_new = Bd	Stores the old Bd.
352.2	solve_H0_lower(...)	Binit(H0)=Lower(H0)+Bd_turn.
352.3	solve_affine_lower(...)	Calculates scale/offset for LowerBmag.
352.4	IsLatched_new = True	Activates the latch.
356	Bd_new = Upper(Hlast) - Blast	Parameter of the new decreasing branch; OnInitMag=False.
358	if OnInitMag_new	B=Binit(H).
360	IsLatched_new and H > H0_new	Affine segment is still active.
364	max(BlatchLower, BlatchLowerLimit)	Bounded affine branch.
362	IsLatched_new = False	Latch cleared.
366	Upper(H)-Bd_new < Binit(H) and H<=0	Checks entry onto the initial path.
368/372	OnInitMag=True; B=Binit(H)	Transition to Binit.
370	B=Upper(H)-Bd_new	Ordinary decreasing minor branch.

Using min on the upper-latched branch and max on the lower-latched branch is not merely cosmetic symmetry; it is a geometric safeguard against crossing the preceding loop.

7.3 main.py Control Loop

The main.py module acts as the top level of FIG. 3: it generates H(t), stores the history state, determines the direction of motion, and calls the corresponding half of the algorithm.

External imports:

```
import numpy as np
import matplotlib.pyplot as plt
from BH import BH_increasing, BH_decreasing
from Hscan import Hscan1, Hscan2
```

Initialization:

```
Hlast = 0.0
Blast = 0.0
OnInitMag = True
IsLatched = False
Bd = 0.0
Bd_turn = 0.0
H0 = 0.0
scale = 0.0
offset = 0.0
```

This start corresponds to the demagnetized state $(H, B) = (0, 0)$ on the initial magnetization path. The check $Hvalues[0] \approx 0$ is an important prerequisite for this initialization.

The variable previous_direction can take the values "inc", "dec", or None. On the first nonzero iteration, the corresponding was_inc/was_dec is set to True so that the start from (0,0) is not interpreted as a reversal. After each actual step, previous_direction is updated.

Programmatic analogue of steps 302-304:

```
if np.isclose(H, Hlast):
    B = Blast
    # previous_direction is NOT changed
```

The state is preserved and the direction is not changed. This behavior is especially important near an extremum of the test $H(t)$, where discretization can produce equal or nearly equal samples.

The Bvalues array has the same length as Hvalues. Initially Bvalues[0]=0. After each iteration the current point is stored, and then Hlast = H and Blast = B. At the end, the array BHloop = column_stack((Hvalues, Bvalues)) is created, after which matplotlib plots the trajectory in time order rather than sorting the points by H. This is essential: hysteresis would disappear if the points were sorted by H.

7.4 Test Functions from the Hscan.py Module

The test function is selected in main.py:

```
from Hscan import Hscan1, Hscan2
```

$$Hscan1(t) = 600 \sin(120\pi t) \exp[-t/(3T)], \quad T = 2\pi/(120\pi) = 1/60 \text{ s}$$

Thus, Hscan1 is a 60 Hz sinusoidal field with an initial amplitude of 600 A/m and an exponential decay time constant of $3T = 0.05 \text{ s}$. Such a signal naturally creates a sequence of loops of decreasing amplitude around zero and is useful for a general check of memory behavior.

$$Hscan2(t) = Hbias(1 - e^{-(\tau/\tau_{bias})}) + A0 e^{-(\tau/\tau_{decay})} \sin(\omega\tau)$$

In the current code, $Hbias = 0.8 \cdot Hc$, $A0 = 1.5 \cdot Hc$, $n_cycles = 5$, $\tau_{bias} = 0.01 \cdot T_{sim}$, and $\tau_{decay} = 0.8 \cdot T_{sim}$, where T_{sim} is the simulation time. The purpose of Hscan2 is to create several reversals around a positive bias and thereby activate the affine-correction branches more frequently.

7.5 Loading Parameters in Parameters.py

Parameters.py opens Parameters.txt once at import, skips blank lines and comments, splits each line at the first “=” character, and stores the float value in the parameters dictionary.

```
parameters = {}

with open(file_path, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        name, value = line.split("=", 1)
        parameters[name.strip()] = float(value.strip())
```

BH.py then executes `from Parameters import parameters as par`, and Bs, Br, and Hc are read from the dictionary once. Therefore, the text file is not reread at every B(H) call. At the same time, this means that modifying Parameters.txt while the process is already running does not automatically update Bs, Br, and Hc in BH.py.

7.6 Correspondence Between the Patent and the Python Implementation

Element	Patent	Current implementation
Upper/Lower/Binit formulas	Literally Table 1	Direct correspondence.
UpperBmag/LowerBmag	Literally Table 1	Direct correspondence.
BlatchUpper/Lower and limits	Literally Table 1	Direct correspondence.
Conditions in steps 310/350, 320/360, 326/366	Literally FIG. 3	Transferred into if-statements.
Bd at reversal	Literally steps 316/356	Direct correspondence.
Storage of Bd_turn	Literally steps 312/352	Direct correspondence.
Numerical method for finding H0	Not specified	Bracket + bisection selected.
Scale/offset formulas	Not written explicitly	Derived from two endpoint conditions.
Representation of dir	Patent: dir	Python: previous_direction + was_inc/was_dec.
General wrapper Find B(H)	Step 300 in the patent	Split in the current project between main.py and the two BH functions.

It is particularly important not to attribute the explicit scale and offset formulas to the patent: the patent requires them to be “solved,” whereas the final expressions in the report and code are our algebraic implementation of that requirement.

7.7 Checks, Tests, and Diagnostic Invariants

The following checks should preferably be automated.

Check	Criterion	What it detects
Basic points	$\text{Upper}(0) \approx \text{Br}$; $\text{Lower}(0) \approx -\text{Br}$; $\text{Binit}(0) \approx 0$	Checks the parameterization of the major loop.
Continuity at reversal, $H \uparrow$	$\text{Lower}(H_{\text{last}}) + \text{Bd_new} = \text{Blast}$	The new branch passes through the turning point.
Continuity at reversal, $H \downarrow$	$\text{Upper}(H_{\text{last}}) - \text{Bd_new} = \text{Blast}$	Mirrored check.
H0 residual, upper	$\text{Binit}(H_0) - \text{BlatchUpperLimit}(H_0) \approx 0$	Root-solver check.
H0 residual, lower	$\text{Binit}(H_0) - \text{BlatchLowerLimit}(H_0) \approx 0$	Root-solver check.
Affine endpoint upper	$\text{BlatchUpper}(H_{\text{last}}) = \text{Blast}$ and $\text{BlatchUpper}(H_0) = \text{Binit}(H_0)$	Check of scale/offset.
Affine endpoint lower	$\text{BlatchLower}(H_{\text{last}}) = \text{Blast}$ and $\text{BlatchLower}(H_0) = \text{Binit}(H_0)$	Check of scale/offset.
Major-loop enclosure	$\text{Lower}(H) \leq B \leq \text{Upper}(H)$	Primary test for the absence of the Chan-type artifact.
Latch upper bound	$B \leq \text{BlatchUpperLimit}(H)$ during upper latch	Do not cross the old branch.
Latch lower bound	$B \geq \text{BlatchLowerLimit}(H)$ during lower latch	Mirrored check.
$H = H_{\text{last}}$	$B = \text{Blast}$, state does not change	Steps 302-304.
Sign symmetry	For symmetrical parameters, the response to a $-H$ history should be the mirror image	Strong regression test.

Automatic check for leaving the major loop:

```

Bupper = Upper(Hcalc)
Blower = Lower(Hcalc)

outside = (Bcalc > Bupper + tol) | (Bcalc < Blower - tol)

if np.any(outside):
    raise RuntimeError("B-H trajectory left the major loop")

```

This invariant directly checks the Chan defect that the patent was created to eliminate.

7.8 Notes on the Current Implementation and Recommendations**Accuracy of np.isclose for detecting stationary H:**

main.py uses np.isclose(H, Hlast) with the default rtol and atol. For determining the field direction, it is better to specify a dedicated absolute tolerance, for example `np.isclose(H, Hlast, rtol=0.0, atol=H\_tol)`, so that the relative tolerance does not depend on the magnitude of H and does not accidentally “swallow” small steps near an extremum.

Initial values of H0, scale, offset, and Bd_turn:

In the current main.py they are initialized to zero. This is acceptable because the corresponding values should not be used when IsLatched=False. For debugging, a stricter option would be None, because accidental use of an uninitialized latch parameter would then be detected immediately. However, the existing architecture already contains None checks inside BH.py, so a transition to None can be made later.

Edge case H0 = Hlast:

solve_H0_upper/lower immediately return Hlast if f(Hlast) is already close to zero. After this, solve_affine_* may obtain an almost zero denominator if the basis function has the same value at Hlast and H0. This is not necessarily an error — geometrically it is a degenerate affine segment — but this case should be covered by a separate test and a decision made as to how it should be treated: as no need for a latch, as a zero-length segment, or as an admissible special branch.

Limiting the H0 search:

Bracketing doubles the step until a sign change is found or max_iter is exhausted. For production code, it is useful to restrict H to a physically reasonable range and include Hlast, Bd_turn, and the values of f at the interval endpoints in the diagnostic message. This would substantially simplify debugging of state errors.

Explicit top-level BH_step():

Conceptually, the patent has a single Find B(H) operation, whereas the current program splits it between main.py and BH_increasing/BH_decreasing. For further development, it would be convenient to introduce a wrapper BH_step(H, state) that performs the H≈Hlast check, direction detection, and the appropriate branch call itself. Then main.py would be responsible only for the time scan, storage, and plotting.

7.9 Calculation Results

The program plots the hysteresis loop for the selected test function H(t) from the Hscan.py module, starting from the fully demagnetized state, and saves it in the BHloop.csv Excel file. Any test function can be created and saved in this module.

```

# =====
# DRAW B(H)
# =====

plt.figure(figsize=(7, 6))

plt.plot(
    BHloop[:, 0],
    BHloop[:, 1],
    linewidth=1.5
)

```

```

plt.xlabel('H (A/m)')
plt.ylabel('B (T)')
plt.title('B-H Hysteresis Loop')

plt.grid(True)

plt.axhline(0.0, linewidth=0.8)
plt.axvline(0.0, linewidth=0.8)

plt.tight_layout()
plt.show()

# =====
# SAVE B-H LOOP TO CSV FILE
# =====

np.savetxt(
    "BHloop.csv",
    BHloop,
    delimiter=",",
    header="H_A_per_m,B_T",
    comments=""
)

```

7.10 Examples of Numerical Hysteresis-Loop Calculations

Below we give examples of hysteresis-loop calculations using our Python implementation of the algorithm.

Parameters.txt:

```

# Magnetic material parameters

Bs=1.6
Br=1.2
Hc=120.0

```

main.py

```

# =====
# GENERATE H(t)
# =====

t_max = 0.1 # time, s
t = np.linspace(0, t_max, 1000)

# Choose a function Hscan(1,2,...) from Hscan.py
Hvalues = Hscan1(t)

```

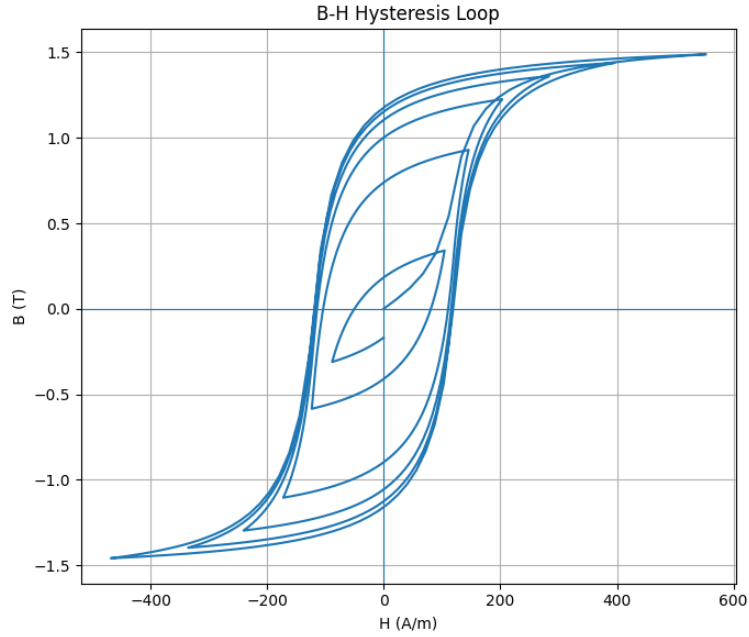
Hscan.py

```

def Hscan1(t):
    w = 120 * pi
    T = (2 * pi) / w
    return 600 * np.sin(w * t) * np.exp(-t / (T * 3))

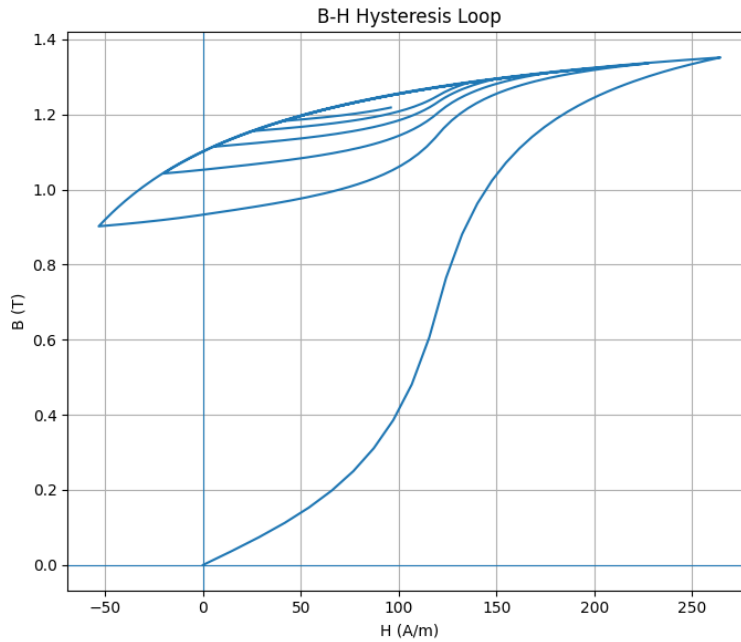
# Testing asymmetrical loops
Hc = par["Hc"]

```



$$H_{scan1}(t) = 600 \sin(120\pi t) \exp[-t/(3T)], \quad T = 2\pi/(120\pi) = 1/60 \text{ s}$$

A 60 Hz sinusoidal field with an initial amplitude of 600 A/m and an exponential decay time constant of $3T = 0.05$ s. Such a signal naturally produces a sequence of loops of decreasing amplitude around zero and is useful for a general check of memory behavior.



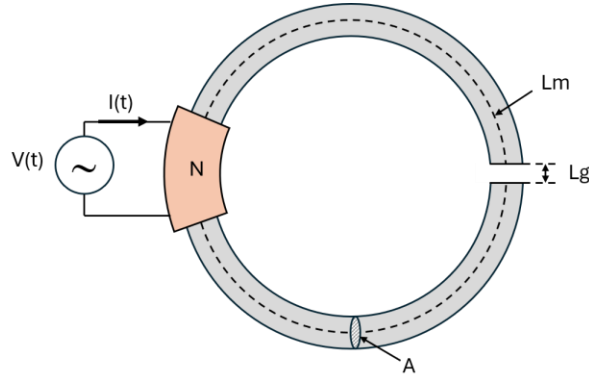
$$H_{scan2}(t) = H_{bias}(1 - e^{-(\tau/\tau_{bias})}) + A_0 e^{-(\tau/\tau_{decay})} \sin(\omega\tau)$$

In the current code, $H_{bias} = 0.8 \cdot H_c$, $A_0 = 1.5 \cdot H_c$, $n_{cycles} = 5$, $\tau_{bias} = 0.01 \cdot T_{sim}$, and $\tau_{decay} = 0.8 \cdot T_{sim}$, where T_{sim} is the simulation time. The purpose of H_{scan2} is to create several reversals around a positive bias and thereby activate the affine-correction branches more frequently.

8. Python Project for Calculating Transient Currents in an Inductor with a Magnetic Core

To clarify the computational algorithm used by LTspice when modelling inductors with ferromagnetic cores, and at the same time to test the developed Python implementation of the magnetic hysteresis model, the basic relations for such inductors and a numerical method for solving the corresponding system of equations are considered below. The fundamentals of magnetic-circuit theory are assumed to be known, so only the provisions directly required for constructing the computational algorithm are presented here. The Python project is stored in the “Chan-Engelhardt-Model-Inductor” folder in the [GitHub](#) repository.[4]

The geometry of the inductor is shown below. The core shape and its uniform cross-section may be arbitrary; a ring has been selected here. A gap in the core is also permitted.



Let us write the equations for the magnetic-field strengths and the current in the coil $I(t)$, which are related to one another:

$$\begin{cases} H_m(t)L_m + H_g(t)L_g = NI(t) \\ RI(t) = V(t) - NA \frac{dB(t)}{dt} \end{cases} \quad (1)$$

where H_m – magnetic-field strength (A/m) inside the magnetic core, H_g – magnetic-field strength in the gap (A/m), R – coil resistance (Ω), $V(t)$ – external EMF source (V), and $-NA \frac{dB(t)}{dt}$ – coil self-induction EMF. With the same cross-section throughout the core, the magnetic flux density $B(t)$ will be uniformly distributed along it due to continuity of its normal component at material boundaries. However, its dependence on magnetic-field strength must be taken into account. We choose $H_m(t)$, as the independent magnetic parameter, so that $B(t) = B(H_m(t))$. Within the gap, one can immediately write $H_g(t) = B(H_m(t))/\mu_0$. As a result, the following system of equations is obtained:

$$\begin{cases} I(t) = \frac{L_m}{N} H_m(t) + \frac{L_g}{\mu_0 N} B(H_m(t)) \\ RI(t) = V(t) - NA \frac{dB(H_m(t))}{dt} \end{cases} \quad (2)$$

Substituting the current from the first equation into the second gives a nonlinear differential equation for the magnetic-field strength, closed by the hysteresis relation $B(H_m(t))$:

$$NA \frac{dB(H_m(t))}{dt} + \frac{RL_m}{N} H_m(t) + \frac{RL_g}{\mu_0 N} B(H_m(t)) = V(t) \quad (3)$$

For the numerical solution of (3), we use the finite-difference [Backward Euler](#) method for the time step $t_{n+1} = t_n + \Delta t$:

$$NA \frac{B(H_{m,n+1}) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H_{m,n+1} + \frac{RL_g}{\mu_0 N} B(H_{m,n+1}) = V_{n+1} \quad (4)$$

where $H_{m,n} = H_m(t_n)$, $H_{m,n+1} = H_m(t_n + \Delta t)$, and $V_{n+1} = V(t_n + \Delta t)$.

For the numerical solution of (4), the residual function is introduced:

$$F(H) = NA \frac{B(H) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H + \frac{RL_g}{\mu_0 N} B(H) - V_{n+1}. \quad (5)$$

The required magnetic-field value is determined from the condition $F(H_{m,n+1}) = 0$. After finding $H_{m,n+1}$ at the current step, the magnetic flux density and current are calculated:

$$B_{n+1} = B(H_{m,n+1}) \quad (6)$$

$$I_{n+1} = \frac{L_m}{N} H_{m,n+1} + \frac{L_g}{\mu_0 N} B(H_{m,n+1}) \quad (7)$$

A special feature of the problem is that the calculation of $B(H)$ is associated with a hysteresis model possessing memory. The state of the model at time t_n can be denoted conventionally by S_n . It includes the current position on the hysteresis loop, the previous direction of motion, information about reversals, parameters of minor loops, and other internal quantities required by the model, which were discussed above when analyzing the algorithm flowcharts. During the solution of the nonlinear equation using the residual $F(H)$ the state S_n must remain unchanged. Each trial field value H_{trial} is treated as an independent hypothesis for the state of the system at time t_{n+1} . For each such value, the complete hysteresis algorithm is executed: $(H_{\text{trial}}, S_n) \rightarrow (B_{\text{trial}}, S_{\text{trial}})$. The model may continue the current hysteresis branch or determine that between steps n and $n+1$ a reversal has occurred, and move to the corresponding new branch. All transitions are performed in exactly the same way as in an ordinary sequential calculation of the hysteresis trajectory. However, the resulting state S_{trial} is only a trial state. It does not replace the state S_n , until the corresponding value H_{trial} satisfies the nonlinear equation. The next trial field value is again calculated starting from the same state S_n : $(H_{\text{trial},1}, S_n) \rightarrow (B_{\text{trial},1}, S_{\text{trial},1})$, $(H_{\text{trial},2}, S_n) \rightarrow (B_{\text{trial},2}, S_{\text{trial},2})$, and so on. A state obtained in one trial iteration must not be used as the initial state for the next trial iteration. Otherwise, the sequence of points generated by the root-finding method would itself become an artificial magnetic history and would alter the hysteresis loop. Thus, during root finding the full hysteresis-model algorithm is executed, but all new parameters returned by it are retained only as trial values. Only after the final value $H_{m,n+1}$ has been found is the corresponding trial state accepted as the new physical state of the system: $S_{n+1} = S_{\text{trial}}$.

To apply the [bisection method](#) when finding the root of the nonlinear equation $F(H) = 0$ an interval $[H_a, H_b]$, must be found at whose endpoints the residual function has opposite signs: $F(H_a)F(H_b) \leq 0$. It is natural to take the known field value at the previous time step as the first boundary: $H_a = H_{m,n}$. At this point $B(H_a) = B(H_{m,n})$, so the derivative term in the residual vanishes. Consequently, $F(H_{m,n}) = RI_n - V_{n+1}$. The quantity $V_{n+1} - RI_n$ determines the physically expected direction of change of magnetic flux density. Indeed, it follows from the electrical equation $NA \frac{dB}{dt} = V - RI$ that when $V_{n+1} - RI_n > 0$ the magnetic flux density tends to increase, whereas when $V_{n+1} - RI_n < 0$ — it tends to decrease. Therefore, the search for the second interval boundary is first carried out in the corresponding direction from $H_{m,n}$. An initial search step ΔH_0 . The first trial boundary is chosen as $H_b = H_{m,n} + s\Delta H_0$, where:

$$s = \begin{cases} +1, & V_{n+1} - RI_n > 0, \\ -1, & V_{n+1} - RI_n < 0. \end{cases}$$

At the point H_b the value $B(H_b)$ is calculated with the state S_n preserved, after which $F(H_b)$. If the condition $F(H_a)F(H_b) \leq 0$, is satisfied, the root is enclosed in the interval found and one can proceed to the bisection method to find it with the specified accuracy and a limit on the maximum number of steps. If the residuals at the endpoints have the same sign, the distance from $H_{m,n}$ to the second boundary is doubled: $\Delta H_0, 2\Delta H_0, 4\Delta H_0, 8\Delta H_0, \dots$. Thus, $H_b^{(j)} = H_{m,n} + s 2^j \Delta H_0$. At each such trial value, an independent hysteresis calculation is again performed from the same preserved state S_n . If an interval with a sign change cannot be found in the physically expected direction, the search is repeated in the opposite direction to improve robustness. The same successive doubling of the distance from $H_{m,n}$. This additional search is particularly useful near reversal points of the hysteresis trajectory, where the simple direction estimate based on $V_{n+1} - RI_n$ may be insufficient.

After an interval $[H_a, H_b]$, satisfying the condition $F(H_a)F(H_b) \leq 0$, has been found, the [bisection method](#). At each iteration the midpoint of the interval is calculated: $H_c = \frac{H_a + H_b}{2}$. For this trial value, starting from the unchanged state S_n , the complete hysteresis-model calculation is performed and $B_c = B(H_c)$, the residual is then calculated $F_c = F(H_c)$. If $F(H_a)F(H_c) \leq 0$, the root lies in the interval $[H_a, H_c]$, and $H_b = H_c$. Otherwise, $F(H_a)F(H_c) > 0$, the root lies in the interval $[H_c, H_b]$, and $H_a = H_c$. The procedure is then repeated. The search terminates when one of the convergence conditions is satisfied: $|F(H_c)| \leq \varepsilon_F$, or $|H_b - H_a| \leq \varepsilon_H$, where ε_F is the allowable residual of the nonlinear equation, and ε_H is the allowable width of the search interval in magnetic-field strength. If the required accuracy is not reached within the specified maximum number of iterations, the time step is regarded as unsolved. In that case, Δt should be reduced, the interval-search parameters changed, or the magnetic and electrical model parameters checked. The last approximation should not be accepted automatically because it may not satisfy the original equation to the required accuracy.

After the root has been found, $H_{m,n+1} = H_c$. At the same time, the values corresponding to this root (6) and (7), are accepted, and the trial state of the hysteresis model becomes the new confirmed state: $S_{n+1} = S_{\text{trial}}$. Only then is the time step regarded as complete, and the state $H_{m,n+1}$, B_{n+1} , I_{n+1} , S_{n+1} is used as the initial state for advancing to the next time instant. The complete algorithm can therefore be represented by the following sequence:

$$\boxed{(H_{m,n}, B_n, I_n, S_n) \rightarrow V_{n+1} \rightarrow \text{поиск интервала} \rightarrow F(H_{m,n+1}) = 0 \rightarrow (H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1})}$$

The fundamental feature of the method is that all trial calculations within the root search are performed with the original hysteresis state S_n unchanged, and the new state is committed only after the nonlinear equation has been solved completely.

8.1 Python Project Structure

The Python project for calculating transient currents in an inductor is a development of the hysteresis-loop calculation project considered in Section 7. The main difference is that the magnetic-field strength H is no longer prescribed externally by the function $H_{\text{scan}}(t)$, but is determined at each time step from the electrical and magnetic circuit equations. The BH.py module implementing the Chan–Engelhardt algorithm is used without modification.

The project structure is:

```
project/
├─ main.py
├─ BH.py
├─ V.py
├─ Parameters.py
└─ Parameters.txt
```

BH.py calculates the magnetic flux density and updates the hysteresis-model state; V.py defines the external voltage; Parameters.py loads parameters from the text file Parameters.txt; main.py organizes the time-stepping loop, solves the nonlinear equation, and saves the results.

8.2 Initial State

The calculation starts from the fully demagnetized state. In main.py, the initial values of the electrical and magnetic variables and the state of the hysteresis model are defined simultaneously:

```
Hlast = 0.0
Blast = 0.0
Ilast = 0.0

OnInitMag = True
IsLatched = False

Bd = 0.0
Bd_turn = 0.0
```

```
H0 = 0.0
scale = 0.0
offset = 0.0
```

```
previous_direction = None
```

Thus, the first point corresponds to the origin on the initial magnetization curve. The variables Hlast, Blast, and llast always refer thereafter to the last confirmed time step.

The time grid is constructed with a constant step dt, and the program additionally checks that the total simulation time is an integer number of time steps:

```
n_steps = int(round(par["simulation_time"] / par["dt"]))

if not np.isclose(
    n_steps * par["dt"],
    par["simulation_time"]
):
    raise ValueError(
        "simulation_time must be an integer multiple of dt"
    )

t = np.arange(n_steps + 1, dtype=float) * par["dt"]
```

8.3 Trial Calculation of the Magnetic State

When solving the nonlinear equation at the next time step, many different trial values of the field Htrial must be checked. For this purpose, the function evaluate_hysteresis_trial() is introduced. First, the direction of field change relative to the last confirmed value is determined:

```
if Htrial > Hlast:
    ...
    result = BH_increasing(...)
or
result = BH_decreasing(...)
```

If the direction of motion has changed relative to the previous time step, the functions BH_increasing() or BH_decreasing() automatically execute all required transitions of the Chan–Engelhardt algorithm, including construction of a new minor branch and, when necessary, affine correction.

It is essential that the result of the call is stored in temporary variables:

```
Btrial = result[0]
trial_state = result[1:]
```

and is not written immediately into the main model state.

In other words, each trial value Htrial is regarded as a possible next state of the system, but remains only a hypothesis until the root search has been completed.

8.4 Preserving the Hysteresis State During Root Finding

The entire root-finding procedure is performed with Hlast, Blast, OnInitMag, IsLatched, Bd, Bd_turn, H0, scale, offset, and previous_direction unchanged; these values correspond to the last confirmed time step. For a particular Htrial, the hysteresis functions may return modified state parameters, but these are stored only in trial_state. The next trial begins again from the same original state. This avoids a fundamental error: the points examined by the root-finding method must not themselves form a fictitious history of magnetic-field variation.

8.5 Residual Function

For each Htrial, the corresponding magnetic flux density is first calculated, followed by the trial current and the residual:

```
def residual(Htrial, Vnext):

    Btrial, trial_state, trial_direction = \
        evaluate_hysteresis_trial(Htrial)

    Itrial = (
        par["l_Fe"] * Htrial
        + (par["g"] / mu0) * Btrial
    ) / par["N"]

    F = (
        par["R"] * Itrial
        + par["N"] * par["A"]
        * (Btrial - Blast) / par["dt"]
        - Vnext
    )

    return F, Btrial, Itrial, trial_state, trial_direction
```

Thus, residual() returns not only the residual required by the numerical solver, but also the complete trial state of the system corresponding to the given Htrial.

8.6 Finding the Bracketing Interval for Bisection

At the beginning of a new time step, the residual is evaluated at the already known point Hlast:

```
F0, B0, I0, state0, direction0 = residual(Hlast, Vnext)
```

The most likely direction in which to search for the root is then determined:

```
drive = Vnext - par["R"] * Ilast

if drive > 0.0:
    primary_direction = 1.0
elif drive < 0.0:
    primary_direction = -1.0
```

The search is first performed in this direction. A trial step `initial_H_step` is taken from `Hlast`, after which the step is successively doubled:

```
Hb = Hlast + search_direction * step
Fb, _, _, _ = residual(Hb, Vnext)

if Fa * Fb <= 0.0:
    bracket_found = True
    break

step *= 2.0
```

As soon as the residual changes sign, an interval containing the root has been found.

To improve robustness, if the interval is not found in the initially expected direction, the entire search is repeated in the opposite direction:

```
for search_direction in (
    primary_direction,
    -primary_direction
):
```

This is particularly useful near reversal points of the hysteresis trajectory.

8.7 Bisection Method

After the interval has been found, the standard bisection procedure begins. At each iteration its midpoint is selected: $H_m = 0.5 * (H_a + H_b)$. The complete trial magnetic state is then recalculated for this value:

```
(
    Fm,
    Bm,
    Im,
    state_m,
    direction_m
) = residual(Hm, Vnext)
```

The half of the interval in which the residual changes sign is then retained:

```
if Fa * Fm <= 0.0:
    Hb = Hm
    Fb = Fm
else:
    Ha = Hm
    Fa = Fm
```

The calculation terminates when the specified accuracy in the residual or in the interval width has been reached:

```
if (
    abs(Fm) <= par["residual_tol"]
    or abs(Hb - Ha) <= par["H_tol"]
):
    return best
```

If the maximum number of iterations is exhausted without reaching the required accuracy, the program raises an error and the last approximation is not automatically accepted as a physical state.

8.8 Committing the New State

Only after the nonlinear equation has been solved successfully does the trial state become the actual state of the system at the new time step:

```
(
    OnInitMag,
    IsLatched,
    Bd,
    Bd_turn,
    H0,
    scale,
    offset
) = state_next
```

The calculated current, field, and magnetic flux density are then stored:

```
Ivalues[n + 1] = Inext
Hvalues[n + 1] = Hnext
Bvalues[n + 1] = Bnext

Hlast = Hnext
Blast = Bnext
Ilast = Inext
```

Only after this does the next time step begin. This provides a fundamental separation between the trial state used inside the nonlinear solver and the confirmed state representing the actual magnetic history of the system.

8.9 Voltage Source and Parameters

Unlike the program in Section 7, where a test function $H(t)$ was prescribed, the inductor model is driven by voltage. Its law is placed in the separate V.py module:

```
def V(t):
    return (
        par["V_offset"]
        + par["V_amplitude"]
        * np.sin(2.0 * np.pi * par["V_frequency"] * t)
    )
```

This makes it possible to change the excitation-voltage waveform later without making any changes to the numerical algorithm for solving the magnetic circuit.

As in the previous project, all numerical parameters are placed in Parameters.txt and loaded through Parameters.py:

```
parameters = {}

with open(file_path, "r", encoding="utf-8") as f:
    for line in f:
        ...
        name, value = line.split("=", 1)
        parameters[name.strip()] = float(value.strip())
```

Thus, the computational modules themselves contain no numerical parameters specific to a particular experiment.

8.10 Calculation Results

During integration, three time sequences are stored: coil current, magnetic-field strength, and magnetic flux density. After the simulation is complete, the current is written to Current.csv:

```
np.savetxt(
    "Current.csv",
    np.column_stack((t, Ivalues)),
    delimiter=",",
    header="time_s,I_A",
    comments=""
)
```

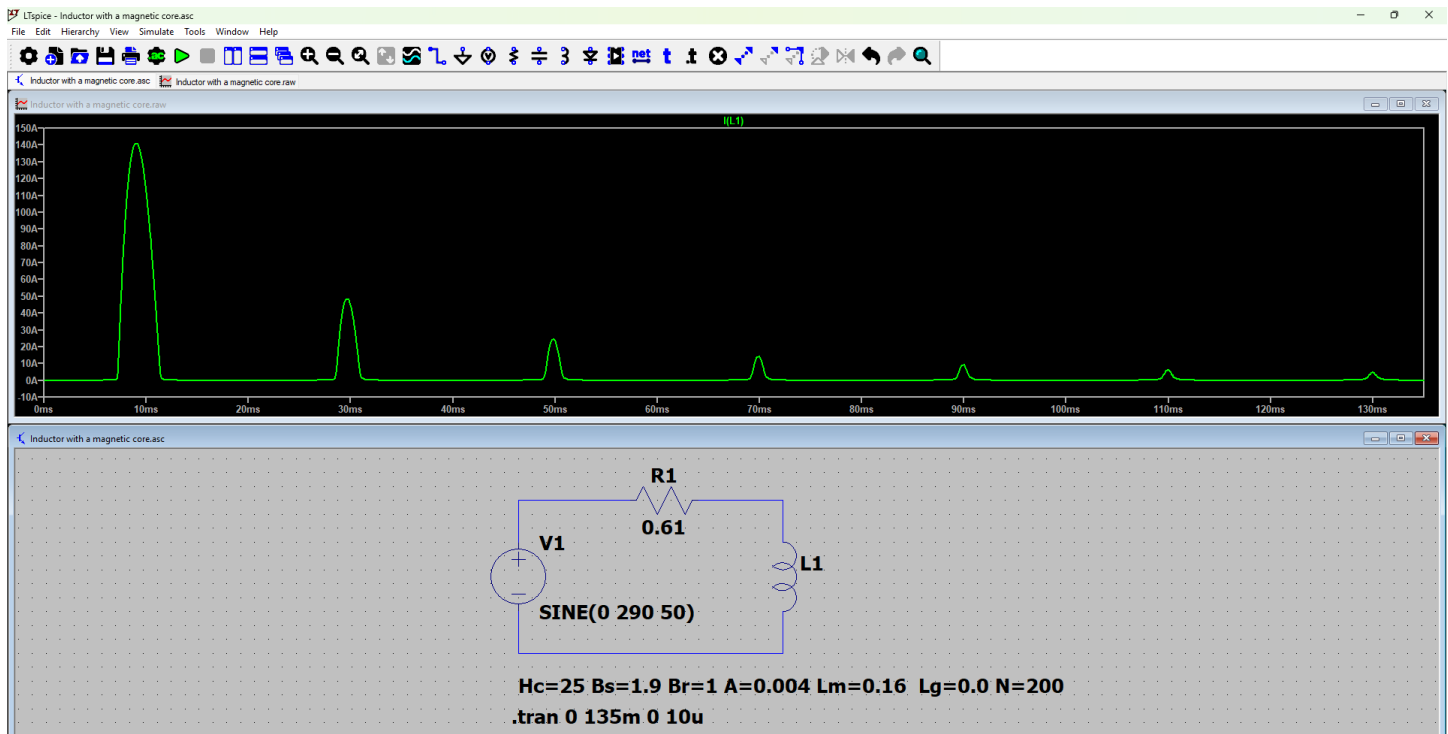
The hysteresis trajectory is saved separately in BHloop.csv:

```
np.savetxt(
    "BHloop.csv",
    np.column_stack((Hvalues, Bvalues)),
    delimiter=",",
    header="H_A_per_m,B_T",
    comments=""
)
```

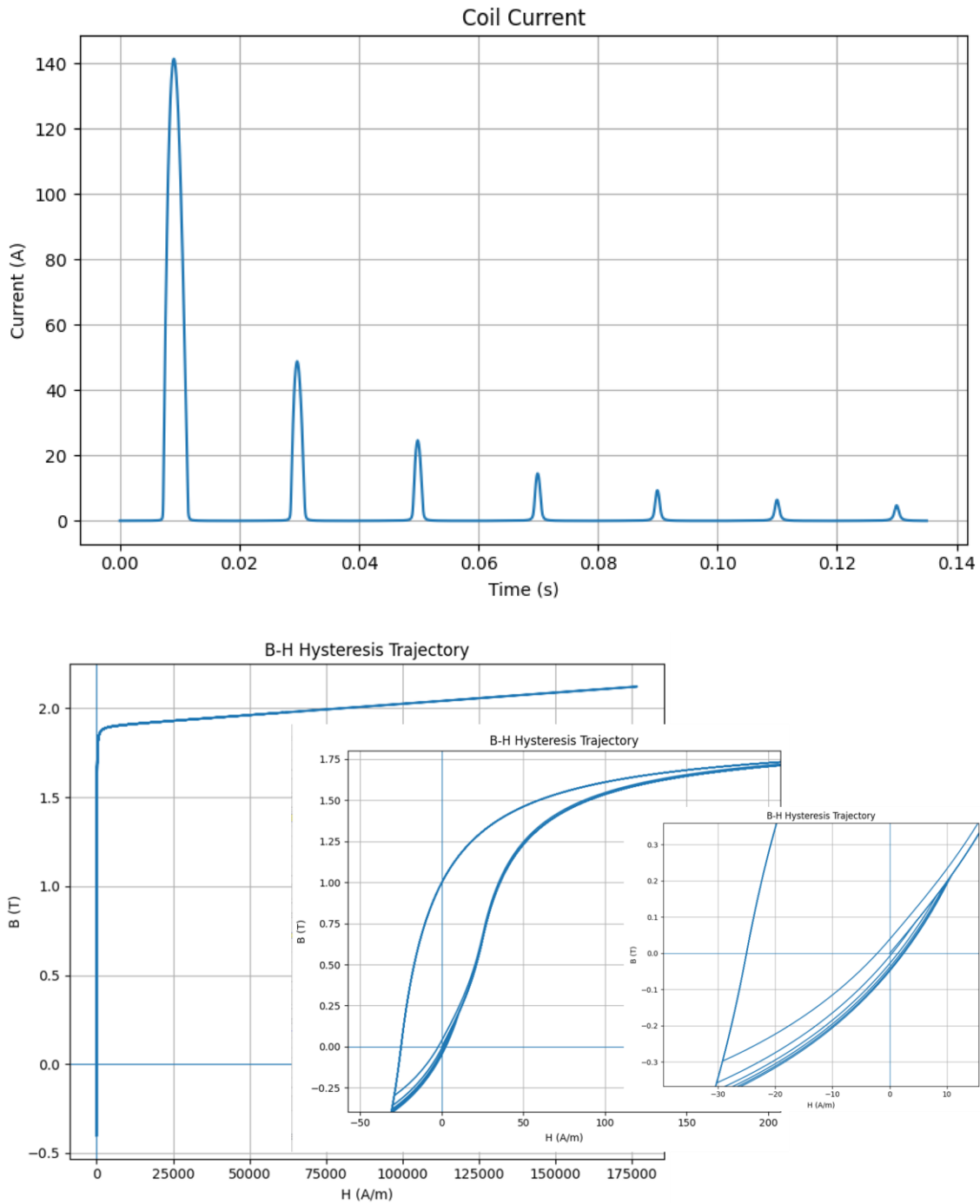
The program also plots $I(t)$ and the complete trajectory $B(H)$. The latter is particularly useful for debugging because, together with the electrical response, it allows one to monitor which major, minor, and asymmetrical branches are selected by the hysteresis model.

8.11 Examples of Inductor Transient-Current Calculations and Comparison with LTspice

The circuit shown below and the transient current in an inductor with an ungapped magnetic core are presented in the patent. We reproduced this circuit in LTspice (folder «LTspice_Inductor with a magnetic core» inside the folder «Chan-Engelhardt-Model-Inductor»).



Our calculation, shown below, is in complete agreement with LTspice.



```
# =====
# MAGNETIC MATERIAL PARAMETERS
# =====

# Saturation magnetic flux density, Bs [T]
Bs=1.9

# Remanent magnetic flux density, Br [T]
Br=1.0

# Coercive magnetic field strength, Hc [A/m]
Hc=25.0
```

```

# =====
# MAGNETIC CORE AND COIL PARAMETERS
# =====

# Number of coil turns, N [-]
N=200

# Coil resistance, R [ohm]
R=0.61

# Mean magnetic path length in the ferromagnetic core, l_Fe [m]
l_Fe=0.16

# Effective magnetic cross-sectional area, A [m^2]
A=0.004

# Air-gap length, g [m]
g=0.0

# =====
# VOLTAGE SOURCE PARAMETERS
# =====

# DC voltage offset [V]
V_offset=0.0

# Sinusoidal voltage amplitude [V]
V_amplitude=290.0

# Voltage frequency [Hz]
V_frequency=50.0

# =====
# SIMULATION PARAMETERS
# =====

# Total simulation time measured from t = 0 [s]
simulation_time=0.135

# Time step [s]
dt=0.000001

# =====
# NONLINEAR SOLVER PARAMETERS
# =====

# Tolerance for the electrical-equation residual F(H) [V]
residual_tol=1.0e-9

# Tolerance for magnetic field strength H [A/m]
H_tol=1.0e-10

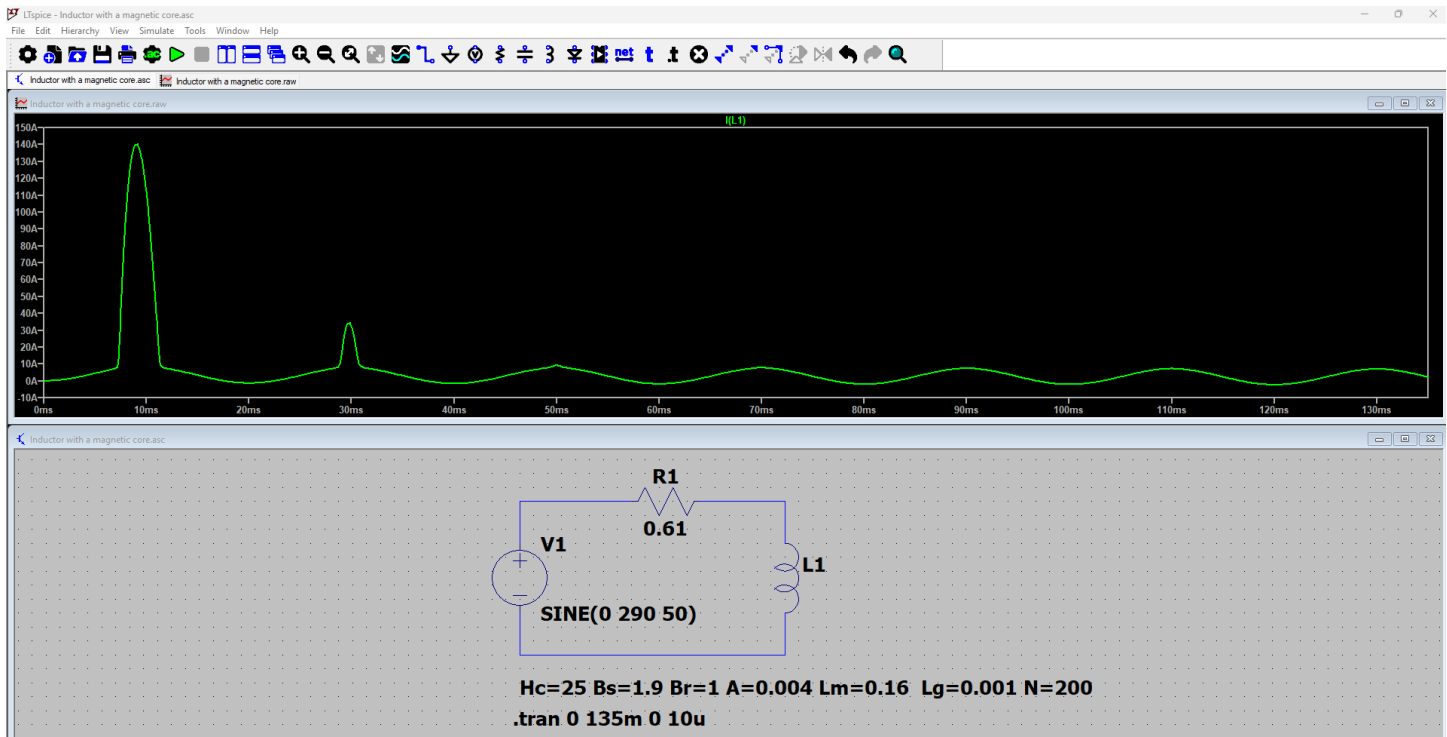
# Maximum number of root-bracketing iterations [-]
max_bracket_iter=80

# Maximum number of bisection iterations [-]
max_bisection_iter=120

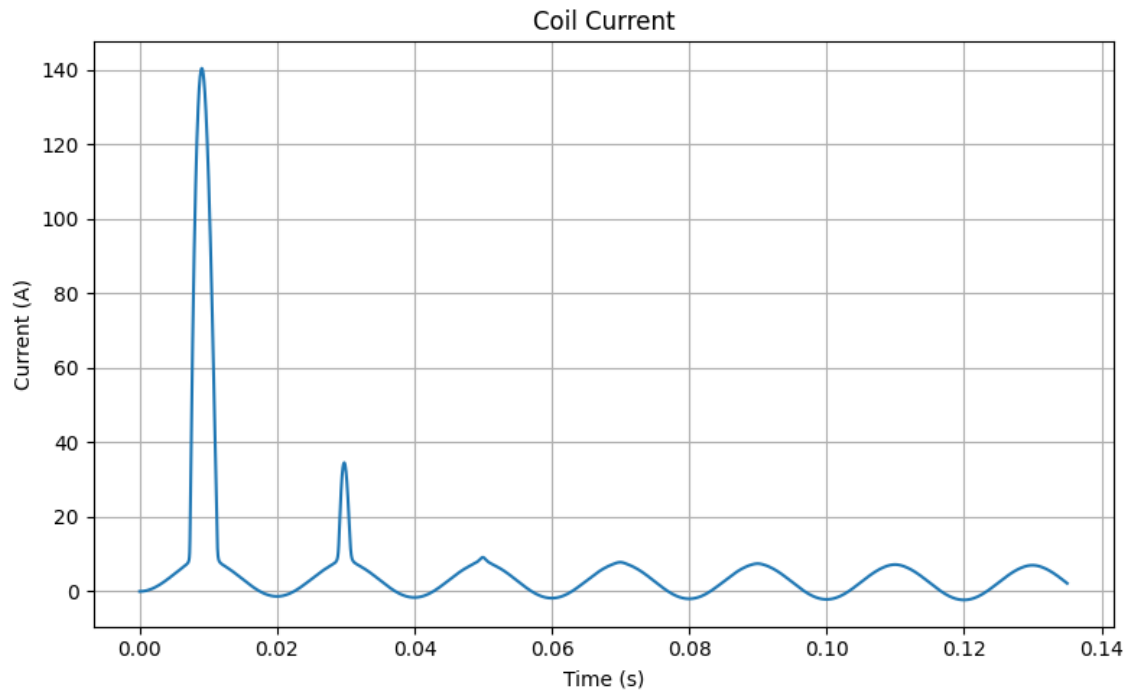
# Initial magnetic-field increment used to search for a root bracket [A/m]
initial_H_step=6.0

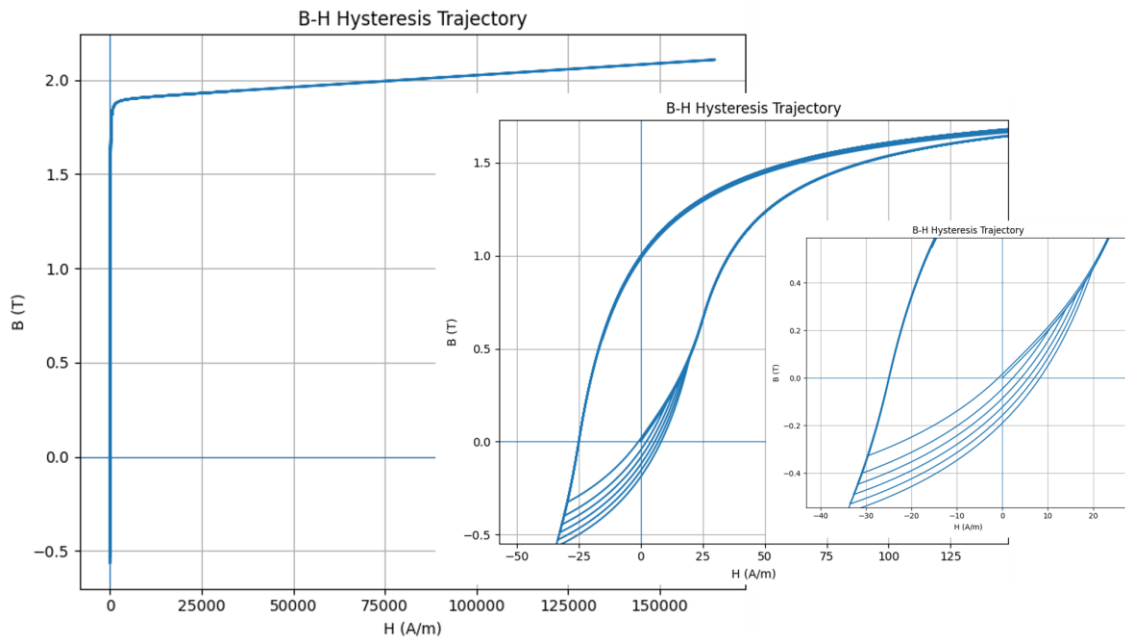
```

We now perform the calculation for the same magnetic inductor, but with a 1 mm gap.



Our calculation, shown below, is in complete agreement with LTspice.





References

- [1] M. Th. Engelhardt, «*Asymmetric Minor Hysteresis Loop Model and Circuit Simulator Including the Same*», U.S. Patent No. 7502723B1, Mar. 10, 2009.
- [2] John H. Chan et al., «*Nonlinear Transformer Model for Circuit Simulation*», IEEE Transactions on Computer-Aided Design, vol. 10, No. 4, APRIL 1991.
- [3] D. C. Jiles and D. L. Atherton, « Theory of ferromagnetic hysteresis», Journal of Magnetism and Magnetic Materials 61 (1986) 48-60.
- [4] Python projects on GitHub: <https://github.com/DmitriyMakhnovskiy/Chan-Engelhardt-Model-BHloops>